```
// ===== backend/src/main/java/com/ihomy/IhomyApplication.java =====
package com.ihomy;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.annotation.EnableScheduling;
/**
 * ihomy 后端启动类:Spring Boot 3 + MyBatis-Plus,
 * 扫描 com.ihomy.mapper(AOP 由 starter 自动装配),开启异步与定时任务支持。
 */
@SpringBootApplication
@MapperScan("com.ihomy.mapper")
@EnableAsync
@EnableScheduling
public class IhomyApplication {
    public static void main(String[] args) {
        SpringApplication.run(IhomyApplication.class, args);
    }
}
// ===== backend/src/main/java/com/ihomy/annotation/OperationLog.java =====
package com.ihomy.annotation;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
 * 操作日志注解:标注在接口方法上,由 OperationLogAspect 自动记录
 * 操作类型/模块/描述,可配置是否保存入参与结果。
 */
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface OperationLog {
    /** 操作描述,如 "新增博客" */
    String description() default "";
    /** 操作类型(LOGIN/LOGOUT/CREATE/UPDATE/DELETE...),默认 OTHER */
    String operationType() default "OTHER";
    /** 所属模块(BLOG/DIARY/ALBUM...),默认 OTHER */
    String module() default "OTHER";
    /** 是否保存方法入参 */
    boolean saveArgs() default true;
    /** 是否保存方法返回值 */
    boolean saveResult() default false;
}
// ===== backend/src/main/java/com/ihomy/annotation/RequirePermission.java =====
package com.ihomy.annotation;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
```

```
/**
 * 权限码注解:标注在接口方法上,value 为权限码(如 "comment:create"),
 * 由 RequirePermissionAspect 在调用前校验。
 */
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RequirePermission {
    /** 所需权限码,须已存在于 sys_auth 并绑定到目标角色 */
    String value();
}
// ===== backend/src/main/java/com/ihomy/aspect/OperationLogAspect.java =====
package com.ihomy.aspect;
import cn.hutool.json.JSONUtil;
import com.ihomy.annotation.OperationLog;
import com.ihomy.common.DictConst;
import com.ihomy.entity.SysOperationLog;
import com.ihomy.security.LoginUser;
import com.ihomy.security.SecurityHelper;
import com.ihomy.service.OperationLogService;
import jakarta.servlet.http.HttpServletRequest;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;
import org.springframework.web.context.request.RequestContextHolder;
import org.springframework.web.context.request.ServletRequestAttributes;
import java.util.Arrays;
/**
 * @OperationLog 注解的切面:环绕记录操作日志—
 * 记录操作人/请求信息/耗时/成败,可选保存入参与结果(截断防超长)。
 */
@Slf4j
@Aspect
@Component
@RequiredArgsConstructor
public class OperationLogAspect {
    private final OperationLogService operationLogService;
    private final SecurityHelper securityHelper;
    /** 请求属性键:标记此请求已被 @OperationLog 切面记录,供 GlobalExceptionHandler 判断避免重复
    补记读接口异常 */
    public static final String OPLOG_RECORDED_ATTR = "com.ihomy.oplog.recorded";
    /** 环绕增强:无论成功失败都落库一条日志,失败时保存错误信息并继续抛出 */
    @Around("@annotation(operationLog)")
    public Object around(ProceedingJoinPoint pjp, OperationLog operationLog) throws Throwable {
        long start = System.currentTimeMillis();
        SysOperationLog logEntry = buildBaseLog(operationLog, pjp);
        Object result = null;
        Throwable thrown = null;
```

```
        try {
            result = pjp.proceed();
            logEntry.setResultStatus(DictConst.LOG_SUCCESS);
            if (operationLog.saveResult() && result != null) {
                String ret = safeJson(result);
                if (ret != null && ret.length() > 2000) {
                    ret = ret.substring(0, 2000);
                }
                logEntry.setDescription(appendResult(logEntry.getDescription(), ret));
            }
        } catch (Throwable e) {
            thrown = e;
            logEntry.setResultStatus(DictConst.LOG_FAILED);
            String msg = e.getMessage();
            if (msg != null && msg.length() > 1000) {
                msg = msg.substring(0, 1000);
            }
            logEntry.setErrorMsg(msg);
            throw e;
        } finally {
            logEntry.setCostTime(System.currentTimeMillis() - start);
            // 业务操作日志双写:落库(原有,运维"操作日志"页)+ server 日志一行(带 tid,
            // 所有 @OperationLog 端点的关键业务节点自动进六要素日志,无需业务代码手动打)
            bizLog(logEntry);
            try {
                operationLogService.save(logEntry);
            } catch (Exception e) {
                OperationLogAspect.log.warn("操作日志切面保存失败: {}", e.getMessage());
            }
            // 标记此请求已由 @OperationLog 记录(含失败),异常传播到 GlobalExceptionHandler 时据
    此跳过补记
            HttpServletRequest req = currentRequest();
            if (req != null) {
                req.setAttribute(OPLOG_RECORDED_ATTR, Boolean.TRUE);
            }
        }
        return result;
    }
    /** server 日志一行:成功 INFO / 失败 WARN(带错误摘要),格式见 docs/日志规范.md */
    private void bizLog(SysOperationLog logEntry) {
        String operator = logEntry.getOperatorId() != null
                ? logEntry.getOperatorName() + "#" + logEntry.getOperatorId() : "anonymous";
        String action = logEntry.getModule() + "." + logEntry.getOperationType()
                + (logEntry.getDescription() == null || logEntry.getDescription().isEmpty()
                        ? "" : " " + logEntry.getDescription());
        if (DictConst.LOG_FAILED.equals(logEntry.getResultStatus())) {
            String err = logEntry.getErrorMsg();
            OperationLogAspect.log.warn("[操作] {} 用户={} 结果=FAILED 耗时={}ms 错误={}",
                    action, operator, logEntry.getCostTime(),
                    err != null && err.length() > 200 ? err.substring(0, 200) : err);
```

```
        } else {
            OperationLogAspect.log.info("[操作] {} 用户={} 结果=SUCCESS 耗时={}ms",
                    action, operator, logEntry.getCostTime());
        }
    }
    /** 组装日志基础信息:注解内容 + 请求 IP/URL + 操作人(登录接口未登录,操作人留空) */
    private SysOperationLog buildBaseLog(OperationLog ann, ProceedingJoinPoint pjp) {
        SysOperationLog logEntry = new SysOperationLog();
        logEntry.setOperationType(ann.operationType());
        logEntry.setModule(ann.module());
        logEntry.setDescription(ann.description());
        logEntry.setTraceId(org.slf4j.MDC.get(com.ihomy.filter.TraceIdFilter.TRACE_ID));
        HttpServletRequest req = currentRequest();
        if (req != null) {
            logEntry.setRequestMethod(req.getMethod());
            logEntry.setRequestUrl(req.getRequestURI());
            logEntry.setIp(com.ihomy.common.Ips.realIp(req));
        }
        LoginUser u = securityHelper.current();
        if (u != null) {
            logEntry.setOperatorId(u.getUserId());
            logEntry.setOperatorName(u.getUsername());
        }
        if (ann.saveArgs()) {
            try {
                String params = safeJson(pjp.getArgs());
                if (params != null && params.length() > 4000) {
                    params = params.substring(0, 4000);
                }
                logEntry.setRequestParams(params);
            } catch (Exception ignored) {
            }
        }
        return logEntry;
    }
    private HttpServletRequest currentRequest() {
        var attrs = RequestContextHolder.getRequestAttributes();
        if (attrs instanceof ServletRequestAttributes sra) {
            return sra.getRequest();
        }
        return null;
    }
    /** 序列化对象为 JSON 并按访问日志同一套清单打码敏感字段,排除请求/响应对象,失败降级 toString
     */
    private String safeJson(Object obj) {
        if (obj == null) return null;
        String raw;
        try {
            if (obj instanceof Object[] arr) {
                raw = JSONUtil.toJsonStr(Arrays.stream(arr)
```

```
                        .filter(x -> !(x instanceof HttpServletRequest)
                                && !(x instanceof jakarta.servlet.http.HttpServletResponse))
                        .toList());
            } else {
                raw = JSONUtil.toJsonStr(obj);
            }
        } catch (Exception e) {
            raw = String.valueOf(obj);
        }
        return com.ihomy.filter.AccessLogFilter.maskJson(raw);
    }
    private String appendResult(String desc, String result) {
        if (desc == null || desc.isEmpty()) return "结果: " + result;
        return desc + " | 结果: " + result;
    }
}
// ===== backend/src/main/java/com/ihomy/aspect/RequirePermissionAspect.java =====
package com.ihomy.aspect;
import com.ihomy.annotation.RequirePermission;
import com.ihomy.common.BizException;
import com.ihomy.common.ResultCode;
import com.ihomy.security.SecurityHelper;
import lombok.RequiredArgsConstructor;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;
/**
 * @RequirePermission 注解的切面:执行前校验当前用户权限码,
 * 无权限直接抛 FORBIDDEN,不进入业务方法。
 */
@Aspect
@Component
@RequiredArgsConstructor
public class RequirePermissionAspect {
    private final SecurityHelper securityHelper;
    /** 拦截所有带 @RequirePermission 的方法,权限不足抛 403 */
    @Around("@annotation(permission)")
    public Object around(ProceedingJoinPoint pjp, RequirePermission permission) throws Throwable
     {
        if (!securityHelper.hasPermission(permission.value())) {
            throw new BizException(ResultCode.FORBIDDEN);
        }
        return pjp.proceed();
    }
}
// ===== backend/src/main/java/com/ihomy/common/AesUtil.java =====
package com.ihomy.common;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
```

```
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.security.SecureRandom;
import java.util.Base64;
/**
 * AES-GCM 加解密工具:用于外挂配置文件中敏感凭证的加密存储。
 *
 * 密文格式:ENC(Base64( iv(12B) || ciphertext || tag(16B) ))
 * 密钥派生:PBKDF2WithHmacSHA256(password=盐值Base64, salt=盐值字节, iterations=100000, keyLen=2
    56bit)
 *
 * 用法:
 *   加密:String cipher = AesUtil.encrypt(plaintext, saltBase64);
 *   解密:String plain   = AesUtil.decrypt(cipher, saltBase64);
 *   外挂文件中写:ENC(xxx),后端读到 ENC 前缀自动解密
 */
public class AesUtil {
    private static final String ALGORITHM = "AES";
    private static final String TRANSFORMATION = "AES/GCM/NoPadding";
    private static final String KEY_DERIVATION = "PBKDF2WithHmacSHA256";
    private static final int ITERATIONS = 100000;
    private static final int KEY_BITS = 256;
    private static final int IV_BYTES = 12;
    private static final int TAG_BITS = 128;
    private static final String ENC_PREFIX = "ENC(";
    private static final String ENC_SUFFIX = ")";
    /** 是否为加密包裹格式 ENC(...) */
    public static boolean isEncrypted(String value) {
        return value != null && value.startsWith(ENC_PREFIX) && value.endsWith(ENC_SUFFIX);
    }
    /** 提取 ENC(...) 内的密文 */
    private static String unwrap(String encValue) {
        return encValue.substring(ENC_PREFIX.length(), encValue.length() - ENC_SUFFIX.length());
    }
    /** 加密并包裹为 ENC(Base64) */
    public static String encrypt(String plaintext, String saltBase64) {
        try {
            byte[] salt = Base64.getDecoder().decode(saltBase64);
            SecretKey key = deriveKey(saltBase64, salt);
            byte[] iv = new byte[IV_BYTES];
            new SecureRandom().nextBytes(iv);
            Cipher cipher = Cipher.getInstance(TRANSFORMATION);
            cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(TAG_BITS, iv));
            byte[] cipherText = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
            ByteBuffer bb = ByteBuffer.allocate(iv.length + cipherText.length);
            bb.put(iv);
```

```
            bb.put(cipherText);
            return ENC_PREFIX + Base64.getEncoder().encodeToString(bb.array()) + ENC_SUFFIX;
        } catch (Exception e) {
            throw new RuntimeException("AES encrypt failed: " + e.getMessage(), e);
        }
    }
    /** 解密 ENC(Base64) 格式的密文;若非加密格式则原样返回(明文兼容) */
    public static String decrypt(String value, String saltBase64) {
        if (!isEncrypted(value)) return value;
        try {
            byte[] salt = Base64.getDecoder().decode(saltBase64);
            SecretKey key = deriveKey(saltBase64, salt);
            byte[] decoded = Base64.getDecoder().decode(unwrap(value));
            ByteBuffer bb = ByteBuffer.wrap(decoded);
            byte[] iv = new byte[IV_BYTES];
            bb.get(iv);
            byte[] cipherText = new byte[bb.remaining()];
            bb.get(cipherText);
            Cipher cipher = Cipher.getInstance(TRANSFORMATION);
            cipher.init(Cipher.DECRYPT_MODE, key, new GCMParameterSpec(TAG_BITS, iv));
            byte[] plain = cipher.doFinal(cipherText);
            return new String(plain, StandardCharsets.UTF_8);
        } catch (Exception e) {
            throw new RuntimeException("AES decrypt failed: " + e.getMessage(), e);
        }
    }
    /** 生成随机盐值(16 字节 Base64) */
    public static String generateSalt() {
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        return Base64.getEncoder().encodeToString(salt);
    }
    /** PBKDF2 派生密钥:password=盐值Base64, salt=盐值字节 */
    private static SecretKey deriveKey(String saltBase64, byte[] salt) throws Exception {
        PBEKeySpec spec = new PBEKeySpec(saltBase64.toCharArray(), salt, ITERATIONS, KEY_BITS);
        SecretKeyFactory skf = SecretKeyFactory.getInstance(KEY_DERIVATION);
        byte[] keyBytes = skf.generateSecret(spec).getEncoded();
        return new SecretKeySpec(keyBytes, ALGORITHM);
    }
}
// ===== backend/src/main/java/com/ihomy/common/AiConst.java =====
package com.ihomy.common;
import java.util.List;
import java.util.Map;
import java.util.Set;
/**
 * AI 模型池与功能绑定常量(V9.48;V9.49 加 LOCAL 本地规则):feature_code 与 model type。
 * 功能只能绑定其允许类型集合内的模型(FEATURE_ALLOWED_TYPES 做绑定校验);
 * 找物/放物允许绑「本地规则(LOCAL)」或「语言模型(LLM)」,实现本地优先 + LLM 兜底。
 */
```

```
public final class AiConst {
    private AiConst() {
    }
    /** 模型类型 */
    public static final String TYPE_LLM = "LLM";
    public static final String TYPE_IMAGE = "IMAGE";
    public static final String TYPE_ASR = "ASR";
    /** 本地规则(内置模型条目,不可删改;零 token 离线解析) */
    public static final String TYPE_LOCAL = "LOCAL";
    /** 服务商/协议:OPENAI=OpenAI 兼容协议(LLM/IMAGE/ASR 默认),BAIDU=百度短语音(仅 ASR) */
    public static final String PROVIDER_OPENAI = "OPENAI";
    public static final String PROVIDER_BAIDU = "BAIDU";
    /** 百度 OAuth access_token 获取地址(API Key + Secret Key 换取) */
    public static final String BAIDU_TOKEN_URL = "https://aip.baidubce.com/oauth/2.0/token";
    /** 百度短语音识别默认 dev_pid(普通话,带标点) */
    public static final String BAIDU_DEFAULT_DEV_PID = "1537";
    /** 有效的服务商(仅模型池新增/更新时校验) */
    public static boolean isValidProvider(String provider) {
        return PROVIDER_OPENAI.equals(provider) || PROVIDER_BAIDU.equals(provider);
    }
    /** 功能 code */
    public static final String FEATURE_ITEM_FIND = "ITEM_FIND";
    public static final String FEATURE_ITEM_PUT = "ITEM_PUT";
    public static final String FEATURE_CHAT = "CHAT";
    public static final String FEATURE_IMAGE = "IMAGE";
    public static final String FEATURE_WEATHER_IMAGE = "WEATHER_IMAGE";
    public static final String FEATURE_ASR = "ASR";
    /** 开源组件升级评估(台账 /ops/oss 升级前 AI 辅助决策) */
    public static final String FEATURE_OSS_UPGRADE_EVAL = "OSS_UPGRADE_EVAL";
    /** 功能 → 允许绑定的模型类型集合(绑定校验用;找物/放物本地或 LLM 二选一) */
    public static final Map<String, Set<String>> FEATURE_ALLOWED_TYPES = Map.of(
            FEATURE_ITEM_FIND, Set.of(TYPE_LOCAL, TYPE_LLM),
            FEATURE_ITEM_PUT, Set.of(TYPE_LOCAL, TYPE_LLM),
            FEATURE_CHAT, Set.of(TYPE_LLM),
            FEATURE_IMAGE, Set.of(TYPE_IMAGE),
            FEATURE_WEATHER_IMAGE, Set.of(TYPE_IMAGE),
            FEATURE_ASR, Set.of(TYPE_ASR),
            FEATURE_OSS_UPGRADE_EVAL, Set.of(TYPE_LLM));
    /** 功能展示顺序(设置页绑定区按此顺序渲染) */
    public static final List<String> FEATURES = List.of(
            FEATURE_ITEM_FIND, FEATURE_ITEM_PUT, FEATURE_CHAT, FEATURE_IMAGE, FEATURE_WEATHER_IM
    AGE, FEATURE_ASR,
            FEATURE_OSS_UPGRADE_EVAL);
    public static boolean isValidFeature(String code) {
        return FEATURE_ALLOWED_TYPES.containsKey(code);
    }
    /** 允许的功能类型集合(绑定校验与前端下拉过滤共用) */
    public static Set<String> allowedTypes(String featureCode) {
        return FEATURE_ALLOWED_TYPES.getOrDefault(featureCode, Set.of());
    }
```

```
    /** 用户可创建的模型类型(LOCAL 为内置保留,不通过 API 创建) */
    public static boolean isValidType(String type) {
        return TYPE_LLM.equals(type) || TYPE_IMAGE.equals(type) || TYPE_ASR.equals(type);
    }
}
// ===== backend/src/main/java/com/ihomy/common/BizException.java =====
package com.ihomy.common;
import lombok.Getter;
/**
 * 业务异常:service 层主动抛出,携带错误码与提示语,
 * 由 GlobalExceptionHandler 统一转换为 Result 失败响应。
 */
@Getter
public class BizException extends RuntimeException {
    private final int code;
    public BizException(ResultCode rc) {
        super(rc.getMessage());
        this.code = rc.getCode();
    }
    /** 带自定义提示语的业务异常(默认码仍由 ResultCode 提供) */
    public BizException(ResultCode rc, String message) {
        super(message);
        this.code = rc.getCode();
    }
}
// ===== backend/src/main/java/com/ihomy/common/DictConst.java =====
package com.ihomy.common;
/**
 * 数据字典常量(V3.9):所有业务状态/类型字段在 DB 中以大写英文单词存储,
 * 与 sys_dict_item 表的 dict_value 一一对应。禁止散落魔法字符串。
 */
public final class DictConst {
    private DictConst() {
    }
    /** 可见范围 visibility */
    public static final String VIS_PRIVATE = "PRIVATE";
    public static final String VIS_MEMBERS = "MEMBERS";
    public static final String VIS_GROUPS = "GROUPS";
    public static final String VIS_FAMILY = "FAMILY";
    public static final String VIS_PUBLIC = "PUBLIC";
    private static final String VIS_FALLBACK = VIS_FAMILY;
    /** 博客状态 blog.status */
    public static final String BLOG_DRAFT = "DRAFT";
    public static final String BLOG_PUBLISHED = "PUBLISHED";
    public static final String BLOG_HIDDEN = "HIDDEN";
    private static final String BLOG_FALLBACK = BLOG_DRAFT;
    /** 用户状态 user_status */
    public static final String USER_ACTIVE = "ACTIVE";
    public static final String USER_DISABLED = "DISABLED";
    private static final String USER_FALLBACK = USER_ACTIVE;
```

```
    /** 角色状态 role_status */
    public static final String ROLE_ENABLED = "ENABLED";
    private static final String ROLE_FALLBACK = ROLE_ENABLED;
    /** 愿望状态 wish_status / video_wish_status */
    public static final String WISH_PENDING = "PENDING";
    public static final String WISH_ACHIEVED = "ACHIEVED";
    public static final String WISH_ABANDONED = "ABANDONED";
    public static final String VWISH_PENDING = "PENDING";
    public static final String VWISH_IMPORTED = "IMPORTED";
    private static final String WISH_FALLBACK = WISH_PENDING;
    /** 放映厅媒体引擎类型 media_server_type */
    public static final String MEDIA_JELLYFIN = "JELLYFIN";
    public static final String MEDIA_EMBY = "EMBY";
    private static final String MEDIA_FALLBACK = MEDIA_JELLYFIN;
    /** 家庭计划状态 plan_status */
    public static final String PLAN_ACTIVE = "ACTIVE";
    public static final String PLAN_DONE = "DONE";
    public static final String PLAN_CANCELLED = "CANCELLED";
    private static final String PLAN_FALLBACK = PLAN_ACTIVE;
    /** 任务状态 task_status */
    public static final String TASK_OPEN = "OPEN";
    public static final String TASK_IN_PROGRESS = "IN_PROGRESS";
    public static final String TASK_REVIEW = "REVIEW";
    public static final String TASK_DONE = "DONE";
    public static final String TASK_CANCELLED = "CANCELLED";
    /** 任务奖励类型 reward_type */
    public static final String REWARD_NONE = "NONE";
    public static final String REWARD_POINTS = "POINTS";
    public static final String REWARD_ITEM = "ITEM";
    private static final String REWARD_FALLBACK = REWARD_NONE;
    /** 入家申请 apply_status */
    public static final String APPLY_PENDING = "PENDING";
    public static final String APPLY_APPROVED = "APPROVED";
    public static final String APPLY_REJECTED = "REJECTED";
    /** 积分订单 order_status */
    public static final String ORDER_PENDING = "PENDING";
    public static final String ORDER_REDEEMED = "REDEEMED";
    /** 提醒周期 reminder_repeat */
    public static final String REPEAT_ONCE = "ONCE";
    public static final String REPEAT_DAILY = "DAILY";
    public static final String REPEAT_WEEKLY = "WEEKLY";
    public static final String REPEAT_MONTHLY = "MONTHLY";
    private static final String REPEAT_FALLBACK = REPEAT_ONCE;
    /** 纪念日周期 ann_recurring */
    public static final String RECUR_ONCE = "ONCE";
    public static final String RECUR_YEARLY = "YEARLY";
    private static final String RECUR_FALLBACK = RECUR_YEARLY;
    /** 记账类型 book_type */
    public static final String BOOK_EXPENSE = "EXPENSE";
    public static final String BOOK_INCOME = "INCOME";
```

```
    public static final String BOOK_TRANSFER = "TRANSFER";
    private static final String BOOK_FALLBACK = BOOK_EXPENSE;
    /** 操作日志结果 log_result */
    public static final String LOG_SUCCESS = "SUCCESS";
    public static final String LOG_FAILED = "FAILED";
    /** 邀请码 invite_status */
    public static final String INVITE_UNUSED = "UNUSED";
    private static final String INVITE_FALLBACK = INVITE_UNUSED;
    /** 菜谱类别 recipe_category */
    public static final String RECIPE_HOT = "HOT";
    public static final String RECIPE_HARD = "HARD";
    public static final String RECIPE_COLD = "COLD";
    public static final String RECIPE_STAPLE = "STAPLE";
    public static final String RECIPE_PORRIDGE = "PORRIDGE";
    public static final String RECIPE_DESSERT = "DESSERT";
    /** 物品类型 item_type */
    public static final String ITEM_KITCHENWARE = "KITCHENWARE";
    public static final String ITEM_INGREDIENT = "INGREDIENT";
    public static final String ITEM_DAILY = "DAILY";
    public static final String ITEM_CLOTHES = "CLOTHES";
    public static final String ITEM_TOOL = "TOOL";
    public static final String ITEM_OTHER = "OTHER";
    /** 电子书格式 book_format */
    public static final String FMT_EPUB = "EPUB";
    public static final String FMT_PDF = "PDF";
    public static final String FMT_TXT = "TXT";
    public static final String FMT_MOBI = "MOBI";
    /** 阅读状态 borrow_status */
    public static final String BORROW_WANT = "WANT_READ";
    public static final String BORROW_READING = "READING";
    public static final String BORROW_FINISHED = "FINISHED";
    /** 开源组件类型 oss_component_type */
    public static final String OSS_TYPE_NPM = "NPM";
    public static final String OSS_TYPE_MAVEN = "MAVEN";
    public static final String OSS_TYPE_SERVICE = "SERVICE";
    /** 开源组件更新类型 oss_update_type */
    public static final String OSS_UPDATE_MAJOR = "MAJOR";
    public static final String OSS_UPDATE_MINOR = "MINOR";
    public static final String OSS_UPDATE_PATCH = "PATCH";
    public static final String OSS_UPDATE_NONE = "NONE";
    /** 开源组件状态 oss_status */
    public static final String OSS_ACTIVE = "ACTIVE";
    public static final String OSS_IGNORED = "IGNORED";
    /** 开源组件集成状态 oss_integration */
    public static final String OSS_INTEG_FULL = "FULL";
    public static final String OSS_INTEG_PARTIAL = "PARTIAL";
    public static final String OSS_INTEG_PLANNED = "PLANNED";
    /** 开源组件管理方 oss_managed_by:RENOVATE 由 Renovate 检测+生成 PR / INTERNAL 台账内部维护
    */
    public static final String OSS_MANAGED_RENOVATE = "RENOVATE";
```

```
    public static final String OSS_MANAGED_INTERNAL = "INTERNAL";
    /** 独立服务部署方式 oss_deploy_type(SERVICE 专用) */
    public static final String OSS_DEPLOY_CONTAINER = "CONTAINER";
    public static final String OSS_DEPLOY_SYSTEMD = "SYSTEMD";
    public static final String OSS_DEPLOY_OTHER = "OTHER";
    /** 独立服务当前版本探测方式 oss_probe_type(SERVICE 专用;空=管理员手工维护当前版本) */
    public static final String OSS_PROBE_NEXTCLOUD = "NEXTCLOUD_STATUS";
    public static final String OSS_PROBE_JELLYFIN = "JELLYFIN_INFO";
    public static final String OSS_PROBE_HA = "HA_CONFIG";
    /* ---------------- 整数(历史 DTO 入参)→ 字典词 转换 ---------------- */
    public static String visibility(Integer v) {
        if (v == null) return VIS_FALLBACK;
        return switch (v) {
            case 0 -> VIS_PRIVATE;
            case 1 -> VIS_MEMBERS;
            case 2 -> VIS_GROUPS;
            case 4 -> VIS_PUBLIC;
            default -> VIS_FAMILY;
        };
    }
    public static String blogStatus(Integer v) {
        if (v == null) return BLOG_FALLBACK;
        return switch (v) {
            case 1 -> BLOG_PUBLISHED;
            case 2 -> BLOG_HIDDEN;
            default -> BLOG_DRAFT;
        };
    }
    public static String wishStatus(Integer v) {
        if (v == null) return WISH_FALLBACK;
        return switch (v) {
            case 1 -> WISH_ACHIEVED;
            case 2 -> WISH_ABANDONED;
            default -> WISH_PENDING;
        };
    }
    public static String planStatus(Integer v) {
        if (v == null) return PLAN_FALLBACK;
        return switch (v) {
            case 1 -> PLAN_DONE;
            case 2 -> PLAN_CANCELLED;
            default -> PLAN_ACTIVE;
        };
    }
    public static String rewardType(Integer v) {
        if (v == null) return REWARD_FALLBACK;
        return switch (v) {
            case 1 -> REWARD_POINTS;
            case 2 -> REWARD_ITEM;
            default -> REWARD_NONE;
```

```
        };
    }
    public static String repeatType(Integer v) {
        if (v == null) return REPEAT_FALLBACK;
        return switch (v) {
            case 1 -> REPEAT_DAILY;
            case 2 -> REPEAT_WEEKLY;
            case 3 -> REPEAT_MONTHLY;
            default -> REPEAT_ONCE;
        };
    }
    public static String annRecurring(Integer v) {
        return v != null && v == 1 ? RECUR_YEARLY : RECUR_FALLBACK;
    }
    public static String bookType(Integer v) {
        if (v == null) return BOOK_FALLBACK;
        return switch (v) {
            case 1 -> BOOK_INCOME;
            case 2 -> BOOK_TRANSFER;
            default -> BOOK_EXPENSE;
        };
    }
    /** 放映厅引擎类型(字符串枚举,非历史整数):未知值一律按 Jellyfin 处理 */
    public static String mediaServerType(String v) {
        return v != null && MEDIA_EMBY.equalsIgnoreCase(v.trim()) ? MEDIA_EMBY : MEDIA_FALLBACK;
    }
    /* ---------------- 字符串(字典词)入参 → 规范化:未知值回退默认,兼容历史数字串 --------------
    -- */
    public static String visibility(String v) {
        if (v == null || v.isBlank()) return VIS_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case VIS_PRIVATE, "0" -> VIS_PRIVATE;
            case VIS_MEMBERS, "1" -> VIS_MEMBERS;
            case VIS_GROUPS, "2" -> VIS_GROUPS;
            case VIS_PUBLIC, "4" -> VIS_PUBLIC;
            default -> VIS_FAMILY;
        };
    }
    public static String blogStatus(String v) {
        if (v == null || v.isBlank()) return BLOG_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case BLOG_PUBLISHED, "1" -> BLOG_PUBLISHED;
            case BLOG_HIDDEN, "2" -> BLOG_HIDDEN;
            default -> BLOG_DRAFT;
        };
    }
    public static String wishStatus(String v) {
        if (v == null || v.isBlank()) return WISH_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case WISH_ACHIEVED, "1" -> WISH_ACHIEVED;
```

```
            case WISH_ABANDONED, "2" -> WISH_ABANDONED;
            default -> WISH_PENDING;
        };
    }
    public static String planStatus(String v) {
        if (v == null || v.isBlank()) return PLAN_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case PLAN_DONE, "1" -> PLAN_DONE;
            case PLAN_CANCELLED, "2" -> PLAN_CANCELLED;
            default -> PLAN_ACTIVE;
        };
    }
    public static String rewardType(String v) {
        if (v == null || v.isBlank()) return REWARD_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case REWARD_POINTS, "1" -> REWARD_POINTS;
            case REWARD_ITEM, "2" -> REWARD_ITEM;
            default -> REWARD_NONE;
        };
    }
    public static String repeatType(String v) {
        if (v == null || v.isBlank()) return REPEAT_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case REPEAT_DAILY, "1" -> REPEAT_DAILY;
            case REPEAT_WEEKLY, "2" -> REPEAT_WEEKLY;
            case REPEAT_MONTHLY, "3" -> REPEAT_MONTHLY;
            default -> REPEAT_ONCE;
        };
    }
    public static String bookType(String v) {
        if (v == null || v.isBlank()) return BOOK_FALLBACK;
        return switch (v.trim().toUpperCase()) {
            case BOOK_INCOME, "1" -> BOOK_INCOME;
            case BOOK_TRANSFER, "2" -> BOOK_TRANSFER;
            default -> BOOK_EXPENSE;
        };
    }
}
// ===== backend/src/main/java/com/ihomy/common/GameConst.java =====
package com.ihomy.common;
/**
 * 小游戏常量:类型/状态/长度上限。
 * 支持 SWF(Flash 播放器)与 GBA(GBA 模拟器)两种导入类型。
 */
public final class GameConst {
    private GameConst() {
    }
    /** 游戏类型 type */
    public static final String TYPE_SWF = "SWF";
    public static final String TYPE_GBA = "GBA";
```

```
    /** 状态 status */
    public static final String STATUS_ACTIVE = "ACTIVE";
    public static final String STATUS_DISABLED = "DISABLED";
    /** 长度上限 */
    public static final int NAME_MAX_LENGTH = 100;
    public static final int DESC_MAX_LENGTH = 500;
    /** 支持的扩展名 */
    public static final String SWF_EXTENSION = ".swf";
    public static final String GBA_EXTENSION = ".gba";
    /** H5 小游戏通关奖励每日上限(防刷:宠物连连看等前端上报分数的游戏按天限次) */
    public static final int PETLINK_DAILY_LIMIT = 5;
}
// ===== backend/src/main/java/com/ihomy/common/GlobalExceptionHandler.java =====
package com.ihomy.common;
import com.ihomy.aspect.OperationLogAspect;
import com.ihomy.entity.SysOperationLog;
import com.ihomy.filter.TraceIdFilter;
import com.ihomy.security.LoginUser;
import com.ihomy.security.SecurityHelper;
import com.ihomy.service.OperationLogService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.ConstraintViolationException;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.slf4j.MDC;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.BindException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.RequestContextHolder;
import org.springframework.web.context.request.ServletRequestAttributes;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;
/**
 * 全局异常处理器:统一捕获控制器抛出的各类异常,
 * 转换为 { code, message } 结构返回,避免异常堆栈直接暴露给前端。
 * 同时补记"无 @OperationLog 注解接口(读接口等)"的异常到操作日志,让运维操作日志页能按 tid 看到读
    接口报错。
 */
@Slf4j
@RestControllerAdvice
@RequiredArgsConstructor
public class GlobalExceptionHandler {
    private final OperationLogService operationLogService;
    private final SecurityHelper securityHelper;
    /** 业务异常:直接透传错误码与提示 */
    @ExceptionHandler(BizException.class)
    public Result<Void> handleBiz(BizException e) {
        log.warn("业务异常: {}", e.getMessage());
```

```
        recordIfNotAnnotated(e.getMessage());
        return Result.fail(e.getCode(), e.getMessage());
    }
    /** @RequestBody 校验失败:取第一个字段错误提示 */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public Result<Void> handleValid(MethodArgumentNotValidException e) {
        String msg = e.getBindingResult().getFieldErrors().stream()
                .findFirst().map(f -> f.getField() + ": " + f.getDefaultMessage())
                .orElse("参数校验失败");
        return Result.fail(ResultCode.BAD_REQUEST.getCode(), msg);
    }
    @ExceptionHandler(BindException.class)
    public Result<Void> handleBind(BindException e) {
        return Result.fail(ResultCode.BAD_REQUEST.getCode(), e.getMessage());
    }
    @ExceptionHandler(ConstraintViolationException.class)
    public Result<Void> handleConstraint(ConstraintViolationException e) {
        return Result.fail(ResultCode.BAD_REQUEST.getCode(), e.getMessage());
    }
    /** 路径/请求参数类型转换失败(如 Long 参数收到 "abc"):400,不再走兜底 500 + 全堆栈 */
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public Result<Void> handleTypeMismatch(MethodArgumentTypeMismatchException e) {
        log.warn("参数类型错误: {}", e.getMessage());
        return Result.fail(ResultCode.BAD_REQUEST.getCode(), "参数类型错误: " + e.getName());
    }
    /** 请求体 JSON 解析失败(格式非法/编码错误):400 而非 500 */
    @ExceptionHandler(HttpMessageNotReadableException.class)
    public Result<Void> handleNotReadable(HttpMessageNotReadableException e) {
        log.warn("请求体解析失败: {}", e.getMessage());
        return Result.fail(ResultCode.BAD_REQUEST.getCode(), "请求体格式错误");
    }
    /** 必填请求参数缺失:400 而非 500 */
    @ExceptionHandler(MissingServletRequestParameterException.class)
    public Result<Void> handleMissingParam(MissingServletRequestParameterException e) {
        log.warn("缺少请求参数: {}", e.getParameterName());
        return Result.fail(ResultCode.BAD_REQUEST.getCode(), "缺少参数: " + e.getParameterName()
    );
    }
    /** 兜底异常:记录完整堆栈后返回 500,避免泄露内部细节 */
    @ExceptionHandler(Exception.class)
    public Result<Void> handleOther(Exception e) {
        log.error("系统异常", e);
        recordIfNotAnnotated(e.getMessage());
        return Result.fail(ResultCode.INTERNAL_ERROR);
    }
    /**
     * 补记无 @OperationLog 注解接口(读接口等)的异常到操作日志。
     * 有 @OperationLog 的写接口异常已由切面记录并打上 OPLOG_RECORDED_ATTR 标记,此处据此跳过,避
    免重复。
     */
```

```
    private void recordIfNotAnnotated(String errorMsg) {
        try {
            HttpServletRequest req = currentRequest();
            if (req == null || Boolean.TRUE.equals(req.getAttribute(OperationLogAspect.OPLOG_REC
    ORDED_ATTR))) {
                return;
            }
            SysOperationLog entry = new SysOperationLog();
            entry.setModule("REQUEST");
            entry.setOperationType(req.getMethod());
            entry.setDescription("接口异常");
            entry.setResultStatus(DictConst.LOG_FAILED);
            entry.setErrorMsg(truncate(errorMsg, 1000));
            entry.setRequestMethod(req.getMethod());
            entry.setRequestUrl(req.getRequestURI());
            entry.setIp(Ips.realIp(req));
            entry.setTraceId(MDC.get(TraceIdFilter.TRACE_ID));
            LoginUser u = securityHelper.current();
            if (u != null) {
                entry.setOperatorId(u.getUserId());
                entry.setOperatorName(u.getUsername());
            }
            operationLogService.save(entry);
        } catch (Exception ex) {
            log.warn("补记接口异常操作日志失败: {}", ex.getMessage());
        }
    }
    private HttpServletRequest currentRequest() {
        var attrs = RequestContextHolder.getRequestAttributes();
        if (attrs instanceof ServletRequestAttributes sra) {
            return sra.getRequest();
        }
        return null;
    }
    private String truncate(String s, int max) {
        if (s == null) return null;
        return s.length() > max ? s.substring(0, max) : s;
    }
}
// ===== backend/src/main/java/com/ihomy/common/HttpUrlUtil.java =====
package com.ihomy.common;
import java.net.InetAddress;
import java.net.URI;
import java.net.UnknownHostException;
import java.util.Locale;
import java.util.Set;
/**
 * 外部服务地址规范化 + 目标限制(SSRF 基本防线):
 * 只放行 http/https、去尾斜杠,并拒绝云元数据端点/链路本地地址/十进制与十六进制 IP 写法。
 * 家庭内网地址(127.0.0.1/192.168.x)对家庭自建服务是刚需,不在拦截范围。
```

```
 */
public final class HttpUrlUtil {
    private static final Set<String> BLOCKED_HOSTS = Set.of(
            "169.254.169.254", "metadata.google.internal", "metadata.goog", "100.100.100.200",
            "192.0.0.192", "fd00:ec2::254", "instance-data");
    private HttpUrlUtil() {
    }
    /** 规范化地址:无协议补 http://,去尾斜杠;不合法/被拦截时抛业务异常(消息由调用方给) */
    public static String normalize(String raw, String errorMessage) {
        return normalize(raw, errorMessage, "这个地址不能作为服务地址,请填家里能访问到的地址");
    }
    /** 同上,拦截目标时用调用方自己的提示语 */
    public static String normalize(String raw, String errorMessage, String blockedMessage) {
        if (raw == null || raw.isBlank()) throw new BizException(ResultCode.BAD_REQUEST, errorMe
    ssage);
        String v = raw.trim();
        String lower = v.toLowerCase(Locale.ROOT);
        if (!lower.startsWith("http://") && !lower.startsWith("https://")) {
            // 带协议但不是 http(s) 的直接判非法,否则会补成 http://ftp://… 这种怪地址
            if (lower.contains("://")) throw new BizException(ResultCode.BAD_REQUEST, errorMessa
    ge);
            v = "http://" + v;
        }
        String bare = v.endsWith("/") ? v.substring(0, v.length() - 1) : v;
        URI uri;
        try {
            uri = URI.create(bare);
            if (uri.getHost() == null || uri.getScheme() == null) throw new IllegalArgumentExcep
    tion("no host");
            if (!"http".equalsIgnoreCase(uri.getScheme()) && !"https".equalsIgnoreCase(uri.getSc
    heme())) {
                throw new IllegalArgumentException("bad scheme");
            }
        } catch (IllegalArgumentException e) {
            throw new BizException(ResultCode.BAD_REQUEST, errorMessage);
        }
        if (isBlockedHost(uri.getHost())) {
            throw new BizException(ResultCode.BAD_REQUEST, blockedMessage);
        }
        return bare;
    }
    /** 云元数据端点/链路本地地址:读得到凭证的那些一律拒绝 */
    public static boolean isBlockedHost(String host) {
        String h = host.toLowerCase(Locale.ROOT);
        if (h.startsWith("[") && h.endsWith("]")) h = h.substring(1, h.length() - 1);
        if (BLOCKED_HOSTS.contains(h) || "0.0.0.0".equals(h)) return true;
        if (h.startsWith("169.254.") || h.startsWith("fe80:")) return true;
        // 十进制/十六进制形式的 IP(169.254.169.254 可写成 2852039166 或 0xA9FEA9FE),当非法地址
        if (h.matches("^\\d+$") || h.matches("^0x[0-9a-f]+$")) return true;
        // IPv4 映射 IPv6(::ffff:169.254.169.254)绕过上面的前缀判断,取出内嵌 IPv4 再判一次
```

```
        int mapped = h.lastIndexOf(":ffff:");
        return mapped >= 0 && isBlockedHost(h.substring(mapped + 6));
    }
    /**
     * 是否落在私有/回环/链路本地/组播网段:抓取用户给的外部地址(如图片直链)时用,
     * 与 isBlockedHost 互补——后者按主机名字符串拦,这里按解析结果拦(域名可指向内网)。
     * 注意家庭自建服务地址(HA/本地模型)走 normalize() 不做此检查,别混用。
     * ponytail:解析与建连之间仍有 DNS 重绑定窗口,家庭场景可接受
     */
    public static boolean isPrivateAddress(InetAddress addr) {
        if (addr.isAnyLocalAddress() || addr.isLoopbackAddress() || addr.isLinkLocalAddress()
                || addr.isSiteLocalAddress() || addr.isMulticastAddress()) {
            return true;
        }
        byte[] b = addr.getAddress();
        if (b.length == 4) {
            // 100.64.0.0/10 共享地址段:云厂商内网与运营商级 NAT(阿里云元数据 100.100.100.200 在
    此段)
            return b[0] == 100 && (b[1] & 0xC0) == 0x40;
        }
        if (b.length != 16) return false;
        if ((b[0] & 0xFE) == 0xFC) return true; // fc00::/7 IPv6 唯一本地地址
        boolean v4mapped = true;
        for (int i = 0; i < 10; i++) {
            if (b[i] != 0) { v4mapped = false; break; }
        }
        if (v4mapped && b[10] == (byte) 0xFF && b[11] == (byte) 0xFF) {
            try {
                // ::ffff:a.b.c.d 映射地址取内嵌 IPv4 再判(192.168.1.1 可这样绕前缀)
                return isPrivateAddress(InetAddress.getByAddress(new byte[]{b[12], b[13], b[14],
     b[15]}));
            } catch (UnknownHostException e) {
                return true; // 4 字节数组恒合法,不会发生;保守判私有
            }
        }
        return false;
    }
}
// ===== backend/src/main/java/com/ihomy/common/Ips.java =====
package com.ihomy.common;
import jakarta.servlet.http.HttpServletRequest;
/**
 * 客户端真实 IP 解析:依次取代理头,取不到回退 remoteAddr。
 * AccessLogFilter 与 OperationLogAspect 共用。
 */
public final class Ips {
    private Ips() {
    }
    public static String realIp(HttpServletRequest req) {
        String ip = req.getHeader("X-Forwarded-For");
```

```
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = req.getHeader("X-Real-IP");
        }
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = req.getHeader("Proxy-Client-IP");
        }
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = req.getRemoteAddr();
        }
        if (ip != null && ip.contains(",")) {
            ip = ip.split(",")[0].trim();
        }
        return ip;
    }
}
// ===== backend/src/main/java/com/ihomy/common/LivePhotoUtil.java =====
package com.ihomy.common;
import java.util.Locale;
/**
 * 苹果 Live Photo 配对工具:实况照片由静态图(HEIC/JPEG)与同名短片(MOV)成对组成,
 * 上传时按文件名基底配对(苹果导出/隔空投送即此命名,如 IMG_1234.HEIC + IMG_1234.MOV)。
 * ponytail:不解析 EXIF/ContentIdentifier,文件名配对已覆盖苹果导出工作流;需处理改名场景再加元数
    据解析。
 */
public final class LivePhotoUtil {
    private LivePhotoUtil() {}
    /** 是否实况短片:按 content-type 或扩展名判定 */
    public static boolean isVideo(String filename, String contentType) {
        if (contentType != null && contentType.toLowerCase(Locale.ROOT).startsWith("video/")) re
    turn true;
        String ext = extension(filename);
        return ext.equals("mov") || ext.equals("mp4") || ext.equals("m4v");
    }
    /** 配对键:去目录、去扩展名、转小写(IMG_1234.HEIC 与 IMG_1234.MOV → img_1234) */
    public static String baseKey(String filename) {
        if (filename == null) return "";
        String name = filename.replace('\\', '/');
        int slash = name.lastIndexOf('/');
        if (slash >= 0) name = name.substring(slash + 1);
        int dot = name.lastIndexOf('.');
        if (dot > 0) name = name.substring(0, dot);
        return name.toLowerCase(Locale.ROOT);
    }
    private static String extension(String filename) {
        if (filename == null) return "";
        int dot = filename.lastIndexOf('.');
        return dot >= 0 ? filename.substring(dot + 1).toLowerCase(Locale.ROOT) : "";
    }
}
// ===== backend/src/main/java/com/ihomy/common/LogAlertAppender.java =====
```

```
package com.ihomy.common;
import ch.qos.logback.classic.spi.IThrowableProxy;
import ch.qos.logback.classic.spi.ILoggingEvent;
import ch.qos.logback.classic.spi.StackTraceElementProxy;
import ch.qos.logback.core.UnsynchronizedAppenderBase;
import com.ihomy.filter.TraceIdFilter;
import org.slf4j.MDC;
/**
 * 同类异常聚合预警的日志采集端(logback appender,见 logback-spring.xml):
 *
 * 只做两件极轻的事—取异常类型/首个应用栈帧 + 把事件交给已注册的 sink(LogAlertService),
 * 自身不做归一化以外的任何 IO;真正落库/发信在 sink 的后台线程完成,不阻塞业务线程。
 *
 * 三条防护:
 *   1. 未注册 sink(Spring 上下文尚未就绪)时直接丢弃,启动期日志不参与聚合;
 *   2. 预警自身的后台线程(名字以 CONSUMER_THREAD_PREFIX 开头)产生的日志不采集,防自噬死循环;
 *   3. append() 内部吞掉一切异常—预警链路出问题绝不能让业务日志打不出来。
 */
public class LogAlertAppender extends UnsynchronizedAppenderBase<ILoggingEvent> {
    /** 预警后台消费者线程名前缀(LogAlertService 用同名线程,用于防自噬) */
    public static final String CONSUMER_THREAD_PREFIX = "ihomy-log-alert";
    /** 判为「应用栈帧」的包前缀:只认业务包,框架/网络栈帧对同类判定没有区分度 */
    private static final String APP_PACKAGE = "com.ihomy.";
    /** 事件接收端(由 LogAlertService 在启动时注册) */
    public interface Sink {
        void onEvent(LogAlertFingerprint.Event event);
    }
    private static volatile Sink sink;
    /** 由 LogAlertService @PostConstruct 注册;传 null 表示停止采集(关闭时) */
    public static void setSink(Sink s) {
        sink = s;
    }
    @Override
    protected void append(ILoggingEvent event) {
        try {
            Sink s = sink;
            if (s == null || event == null) {
                return;
            }
            String thread = Thread.currentThread().getName();
            if (thread != null && thread.startsWith(CONSUMER_THREAD_PREFIX)) {
                return;
            }
            IThrowableProxy tp = event.getThrowableProxy();
            s.onEvent(new LogAlertFingerprint.Event(
                    event.getLevel() == null ? null : event.getLevel().toString(),
                    event.getLoggerName(),
                    event.getFormattedMessage(),
                    tp == null ? null : tp.getClassName(),
                    tp == null ? null : tp.getMessage(),
```

```
                    firstAppFrame(tp),
                    MDC.get(TraceIdFilter.TRACE_ID)));
        } catch (Throwable ignored) {
            // 见类注释第 3 条:不得影响业务日志
        }
    }
    /** 首个应用栈帧(类#方法):取最靠近抛出点的一帧,不含行号以免改代码换行就重新预警 */
    private static String firstAppFrame(IThrowableProxy tp) {
        if (tp == null) {
            return null;
        }
        try {
            StackTraceElementProxy[] frames = tp.getStackTraceElementProxyArray();
            if (frames == null) {
                return null;
            }
            for (StackTraceElementProxy p : frames) {
                StackTraceElement el = p.getStackTraceElement();
                if (el == null || el.getClassName() == null) {
                    continue;
                }
                if (el.getClassName().startsWith(APP_PACKAGE)) {
                    return el.getClassName() + "#" + el.getMethodName();
                }
            }
        } catch (Throwable ignored) {
            // 栈帧解析失败退回「无栈帧」,仍可按异常类型聚合
        }
        return null;
    }
}
// ===== backend/src/main/java/com/ihomy/common/LogAlertFingerprint.java =====
package com.ihomy.common;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Locale;
import java.util.regex.Pattern;
/**
 * 日志「同类问题」归一化(纯函数,不依赖 Spring/logback,便于单测):
 *
 * 判定同一类问题的口径——
 *   - 带异常:级别 + logger + 异常类型 + 首个应用栈帧(类#方法)。故意不含异常消息与 URL 等易变内
    容,
 *     这样「一批不同组件的 Maven 探测超时」会归为同一类,而不是散成几十条各一次的噪声。
 *   - 无异常:级别 + logger + 归一化消息(数字/十六进制/UUID/追踪号一律压成 #,消除易变值)。
 *
 * 归一化后取 SHA-256 十六进制作为指纹 key(稳定、跨重启一致,可直接落库比较)。
 */
public final class LogAlertFingerprint {
```

```
    /** 一条待聚合的日志事件(appender 采集,不含任何 Spring 依赖) */
    public record Event(String level, String logger, String message,
                        String throwableClass, String throwableMessage,
                        String appFrame, String traceId) {
    }
    /** 聚合结果:key 为同类问题的稳定指纹,title 为可读摘要 */
    public record Fingerprint(String key, String title) {
    }
    private static final Pattern UUID_RE =
            Pattern.compile("[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-f
    A-F]{12}");
    private static final Pattern HEX_RE = Pattern.compile("\\b[0-9a-fA-F]{8,}\\b");
    private static final Pattern DIGIT_RE = Pattern.compile("\\d+");
    private static final Pattern SPACE_RE = Pattern.compile("\\s+");
    /** 消息归一化上限(超过部分截断,避免超长 SQL/JSON 撑大指纹) */
    private static final int NORM_MAX = 200;
    /** 摘要上限 */
    private static final int TITLE_MAX = 220;
    private static final String UNKNOWN = "(未识别日志)";
    private LogAlertFingerprint() {
    }
    public static Fingerprint of(Event e) {
        String level = blankTo(e.level(), "INFO");
        String logger = blankTo(e.logger(), "-");
        if (e.throwableClass() != null && !e.throwableClass().isBlank()) {
            String frame = e.appFrame() == null ? "" : e.appFrame();
            String key = hash(level + "|" + logger + "|" + e.throwableClass() + "|" + frame);
            StringBuilder title = new StringBuilder(shortName(e.throwableClass()));
            String msg = oneLine(e.throwableMessage());
            if (!msg.isEmpty()) {
                title.append(": ").append(msg);
            }
            if (!frame.isEmpty()) {
                title.append(" @ ").append(frameShort(frame));
            }
            return new Fingerprint(key, truncate(title.toString(), TITLE_MAX));
        }
        String norm = normalize(e.message());
        if (norm.isEmpty()) {
            norm = UNKNOWN;
        }
        return new Fingerprint(hash(level + "|" + logger + "|" + norm), truncate(norm, TITLE_MAX
    ));
    }
    /** 归一化消息:压平空白 → 抹掉 UUID/长十六进制/数字串(时间、耗时、ID、tid 等易变值) */
    public static String normalize(String message) {
        if (message == null) {
            return "";
        }
        String s = oneLine(message);
```

```
        if (s.isEmpty()) {
            return "";
        }
        s = UUID_RE.matcher(s).replaceAll("#");
        s = HEX_RE.matcher(s).replaceAll("#");
        s = DIGIT_RE.matcher(s).replaceAll("#");
        return truncate(s, NORM_MAX);
    }
    /** 取首行并压平连续空白(堆栈、多行消息只留首行做指纹) */
    private static String oneLine(String s) {
        if (s == null) {
            return "";
        }
        int nl = s.indexOf('\n');
        String first = nl < 0 ? s : s.substring(0, nl);
        return SPACE_RE.matcher(first).replaceAll(" ").trim();
    }
    /** 异常类名取简名:java.net.SocketTimeoutException → SocketTimeoutException */
    public static String shortName(String className) {
        int dot = className.lastIndexOf('.');
        return dot < 0 ? className : className.substring(dot + 1);
    }
    /** 栈帧简写:com.ihomy.common.ThirdPartyHttp#request → ThirdPartyHttp#request */
    public static String frameShort(String frame) {
        if (frame == null || frame.isEmpty()) {
            return "";
        }
        int hash = frame.indexOf('#');
        String cls = hash < 0 ? frame : frame.substring(0, hash);
        int dot = cls.lastIndexOf('.');
        return (dot < 0 ? cls : cls.substring(dot + 1)) + (hash < 0 ? "" : frame.substring(hash)
    );
    }
    private static String hash(String source) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] out = md.digest(source.getBytes(StandardCharsets.UTF_8));
            StringBuilder sb = new StringBuilder(out.length * 2);
            for (byte b : out) {
                sb.append(Character.forDigit((b >> 4) & 0xF, 16)).append(Character.forDigit(b &
    0xF, 16));
            }
            return sb.toString().toLowerCase(Locale.ROOT);
        } catch (NoSuchAlgorithmException e) {
            // SHA-256 是 JDK 必备算法,不会走到;退化用来源字符串本身保证仍能分组
            return Integer.toHexString(source.hashCode());
        }
    }
    private static String blankTo(String v, String fallback) {
        return v == null || v.isBlank() ? fallback : v;
```

```
    }
    private static String truncate(String s, int max) {
        return s.length() <= max ? s : s.substring(0, max);
    }
}
// ===== backend/src/main/java/com/ihomy/common/Loggers.java =====
package com.ihomy.common;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
 * 通用日志工具:三类日志文件的 logger 入口。
 *
 * - access():     客户端调接口(AccessLogFilter 已自动覆盖全部 HTTP 请求;
 *                 WebSocket 消息、其他自定义入口需手动补一条)
 * - thirdParty(): 出站调用第三方服务 API(统一走 common/ThirdPartyHttp,
 *                 特殊流式调用手动补日志)
 * - 其余业务日志: 直接用 @Slf4j 的 log,自动进 server 日志文件
 *
 * 六要素:时间 级别 线程 [tid] 位置(logger) 内容。
 * 级别标准:ERROR=需人工介入(带堆栈)/WARN=可自动恢复但需关注/INFO=关键业务节点/DEBUG=调试细节。
 */
public final class Loggers {
    public static final String ACCESS_NAME = "ACCESS";
    public static final String THIRD_PARTY_PREFIX = "thirdparty.";
    private Loggers() {
    }
    /** 客户端接口调用日志(access 文件) */
    public static Logger access() {
        return LoggerFactory.getLogger(ACCESS_NAME);
    }
    /** 第三方服务出站调用日志(thirdparty 文件);service 如 weather/baidu/ipapi/bing */
    public static Logger thirdParty(String service) {
        return LoggerFactory.getLogger(THIRD_PARTY_PREFIX + service);
    }
}
// ===== backend/src/main/java/com/ihomy/common/OssVersionUtil.java =====
package com.ihomy.common;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
/**
 * 开源组件版本工具:纯规则解析比较,不调用 AI/LLM。
 * 支持形如 "1.2.3" / "v1.2.3" / "1.2.3-SNAPSHOT" / "10.9.0+deb" / "2024.10.0" 的版本号,
 * 比较大小并归类 MAJOR/MINOR/PATCH/NONE。
 */
public final class OssVersionUtil {
    private static final Pattern DIGIT = Pattern.compile("\\d+");
    private OssVersionUtil() {
    }
```

```
    /** 提取数字段:1.2.3 → [1,2,3];无数字返回空列表 */
    public static List<Integer> parse(String version) {
        List<Integer> seg = new ArrayList<>();
        if (version == null || version.isBlank()) {
            return seg;
        }
        Matcher m = DIGIT.matcher(version);
        while (m.find() && seg.size() < 4) {
            seg.add(Integer.parseInt(m.group()));
        }
        return seg;
    }
    /** a > b 返回 1,a < b 返回 -1,相等 0;任一方不可比较返回 null */
    public static Integer compare(String a, String b) {
        List<Integer> sa = parse(a);
        List<Integer> sb = parse(b);
        if (sa.isEmpty() || sb.isEmpty()) {
            return null;
        }
        int n = Math.max(sa.size(), sb.size());
        for (int i = 0; i < n; i++) {
            int va = i < sa.size() ? sa.get(i) : 0;
            int vb = i < sb.size() ? sb.get(i) : 0;
            if (va != vb) {
                return va > vb ? 1 : -1;
            }
        }
        return 0;
    }
    /** 归类更新类型:current → latest;latest 不高于 current 或不可比较时返回 NONE */
    public static String updateType(String current, String latest) {
        List<Integer> sc = parse(current);
        List<Integer> sl = parse(latest);
        if (sc.isEmpty() || sl.isEmpty()) {
            return DictConst.OSS_UPDATE_NONE;
        }
        Integer cmp = compare(latest, current);
        if (cmp == null || cmp <= 0) {
            return DictConst.OSS_UPDATE_NONE;
        }
        if (sl.get(0) > sc.get(0)) {
            return DictConst.OSS_UPDATE_MAJOR;
        }
        int cMinor = sc.size() > 1 ? sc.get(1) : 0;
        int lMinor = sl.size() > 1 ? sl.get(1) : 0;
        if (lMinor > cMinor) {
            return DictConst.OSS_UPDATE_MINOR;
        }
        return DictConst.OSS_UPDATE_PATCH;
    }
```

```
}
// ===== backend/src/main/java/com/ihomy/common/PlantConst.java =====
package com.ihomy.common;
import java.util.Set;
/**
 * 植物养殖常量(V9.67→V9.68):品种/阶段/成长时长/照料加成/冷却/营养/天气/积分。
 * 阶段、营养均由「种植耗时 + 照料加成 + 营养/天气修正」实时推导,不落库。
 */
public final class PlantConst {
    private PlantConst() {
    }
    /** 品种 species */
    public static final String SPECIES_SUNFLOWER = "SUNFLOWER";
    public static final String SPECIES_ROSE = "ROSE";
    public static final String SPECIES_SUCCULENT = "SUCCULENT";
    public static final Set<String> SPECIES = Set.of(SPECIES_SUNFLOWER, SPECIES_ROSE, SPECIES_SU
    CCULENT);
    /** 成长阶段 stage(顺序固定) */
    public static final String STAGE_SEED = "SEED";
    public static final String STAGE_SPROUT = "SPROUT";
    public static final String STAGE_SEEDLING = "SEEDLING";
    public static final String STAGE_BUD = "BUD";
    public static final String STAGE_FLOWER = "FLOWER";
    public static final String STAGE_FRUIT = "FRUIT";
    public static final String[] STAGES = {STAGE_SEED, STAGE_SPROUT, STAGE_SEEDLING, STAGE_BUD,
    STAGE_FLOWER, STAGE_FRUIT};
    /** 各阶段所需累计成长分钟阈值(与 STAGES 一一对应):0h/6h/24h/48h/72h/96h */
    private static final long[] THRESHOLDS = {0, 360, 1440, 2880, 4320, 5760};
    /** 每次照料(浇水/晒太阳)的基础成长加成(分钟) */
    public static final long CARE_BOOST_MINUTES = 120;
    /** 每次施肥的成长加成(分钟) */
    public static final long FERTILIZE_BOOST_MINUTES = 360;
    /** 浇水/晒太阳的家庭级冷却(分钟) */
    public static final long CARE_COOLDOWN_MINUTES = 360;
    /** 施肥的家庭级冷却(分钟) */
    public static final long FERTILIZE_COOLDOWN_MINUTES = 720;
    /** 缺水/枯萎阈值(分钟):超过未浇水即提示 */
    public static final long THIRSTY_MINUTES = 48L * 60;
    public static final long WILT_MINUTES = 72L * 60;
    /** 营养:施肥后从 100 线性衰减到 20(用时 NUTRIENT_DECAY_HOURS);从未施肥的基线为 50 */
    public static final int NUTRIENT_MAX = 100;
    public static final int NUTRIENT_FLOOR = 20;
    public static final int NUTRIENT_BASELINE = 50;
    public static final long NUTRIENT_DECAY_HOURS = 72;
    /** 积分:施肥消耗(扣分,消费侧;照料/收获的获取分值已由 PointsRuleConst 家庭级规则接管) */
    public static final int FERTILIZE_POINTS_COST = 15;
    public static boolean isSpecies(String species) {
        return species != null && SPECIES.contains(species.toUpperCase());
    }
    public static long[] thresholds() {
```

```
    return THRESHOLDS.clone();
}
public static String stageForGrowthMinutes(long minutes) {
    String stage = STAGE_SEED;
    for (int i = 0; i < THRESHOLDS.length; i++) {
        if (minutes >= THRESHOLDS[i]) {
            stage = STAGES[i];
        }
    }
    return stage;
}
public static int stageIndex(String stage) {
    for (int i = 0; i < STAGES.length; i++) {
        if (STAGES[i].equals(stage)) {
            return i;
        }
    }
    return 0;
}
/** 到下一阶段还需的成长分钟(已到最终阶段返回 0) */
public static long minutesToNextStage(long growthMinutes) {
    int idx = stageIndex(stageForGrowthMinutes(growthMinutes));
    if (idx >= THRESHOLDS.length - 1) {
        return 0;
    }
    return Math.max(0, THRESHOLDS[idx + 1] - growthMinutes);
}
/**
 * 营养等级(0-100):从未施肥为基线 50;施肥后从 100 随小时线性衰减到 FLOOR。
 */
public static int nutrientLevel(int fertilizeCount, java.time.LocalDateTime lastFertilizedAt
) {
    if (fertilizeCount <= 0 || lastFertilizedAt == null) {
        return NUTRIENT_BASELINE;
    }
    long hours = Math.max(0, java.time.Duration.between(lastFertilizedAt, java.time.LocalDat
eTime.now()).toMinutes() / 60);
    long decay = NUTRIENT_MAX - hours * (NUTRIENT_MAX - NUTRIENT_FLOOR) / NUTRIENT_DECAY_HOU
RS;
    return (int) Math.max(NUTRIENT_FLOOR, Math.min(NUTRIENT_MAX, decay));
}
/** 营养 → 成长速率倍率(乘到时间累计):0→0.7 / 50→1.0 / 100→1.3 */
public static double nutrientRate(int nutrient) {
    return 0.7 + nutrient * 0.006;
}
/**
 * 天气 → 照料加成倍率(乘到浇水/晒太阳的加成上)。
 * 晴天使劲晒、雨天浇水事半功倍;阴雨/严寒晒太阳效果打折。
 */
public static double weatherCareFactor(String condition, boolean isWater) {
```

```
        if (condition == null) {
            return 1.0;
        }
        return switch (condition) {
            case "clear" -> isWater ? 1.0 : 1.5;
            case "cloud" -> 1.0;
            case "rain" -> isWater ? 1.5 : 0.5;
            case "snow" -> isWater ? 0.8 : 0.3;
            case "fog" -> isWater ? 0.9 : 0.7;
            case "thunder" -> isWater ? 1.3 : 0.4;
            default -> 1.0;
        };
    }
    /** 当前天气是否视为「自动补水」(雨天/雷雨/下雪时植物不会因缺水枯萎) */
    public static boolean isRainWatering(String condition) {
        return "rain".equals(condition) || "thunder".equals(condition) || "snow".equals(conditio
    n);
    }
}
// ===== backend/src/main/java/com/ihomy/common/PointsRuleConst.java =====
package com.ihomy.common;
import java.util.List;
/**
 * 积分获取规则常量:家庭级「哪些功能能得积分、得多少」的 feature code 清单与默认值。
 * 与积分商城(消费侧)不同,这里只管获取侧;每功能家长可设 enabled 开关 + points 分值。
 * 缺行读取时回退这里的默认值,保证升级后行为不变(默认值与原硬编码常量一致)。
 * 任务(TASK)只受开关控制,分值由任务自身 reward_points 决定(hasPoints=false)。
 */
public final class PointsRuleConst {
    private PointsRuleConst() {
    }
    /** feature code */
    public static final String CHECKIN = "checkin";
    public static final String BLOG = "blog";
    public static final String DIARY = "diary";
    public static final String PHOTO = "photo";
    public static final String VIDEO = "video";
    public static final String TASK = "task";
    public static final String GAME_PETLINK = "game_petlink";
    public static final String PLANT_WATER = "plant_water";
    public static final String PLANT_SUN = "plant_sun";
    public static final String PLANT_HARVEST = "plant_harvest";
    /** 分组(前端分区渲染用) */
    public static final String GROUP_CONTENT = "content";
    public static final String GROUP_TASK = "task";
    public static final String GROUP_GAME = "game";
    public static final String GROUP_GARDEN = "garden";
    /**
     * 功能定义:code / 分组 / 默认分值 / 是否可配分值。
     * hasPoints=false 的功能只受开关控制(如任务,分值由任务自身字段决定)。
```

```
     */
    public record Feature(String code, String group, int defaultPoints, boolean hasPoints) {
    }
    /** 功能展示顺序(配置页按此顺序分组渲染) */
    public static final List<Feature> FEATURES = List.of(
            new Feature(CHECKIN, GROUP_CONTENT, 5, true),
            new Feature(BLOG, GROUP_CONTENT, 10, true),
            new Feature(DIARY, GROUP_CONTENT, 8, true),
            new Feature(PHOTO, GROUP_CONTENT, 2, true),
            new Feature(VIDEO, GROUP_CONTENT, 15, true),
            new Feature(TASK, GROUP_TASK, 0, false),
            new Feature(GAME_PETLINK, GROUP_GAME, 10, true),
            new Feature(PLANT_WATER, GROUP_GARDEN, 2, true),
            new Feature(PLANT_SUN, GROUP_GARDEN, 2, true),
            new Feature(PLANT_HARVEST, GROUP_GARDEN, 30, true));
    public static Feature feature(String code) {
        for (Feature f : FEATURES) {
            if (f.code().equals(code)) {
                return f;
            }
        }
        return null;
    }
    public static boolean isValidFeature(String code) {
        return feature(code) != null;
    }
}
// ===== backend/src/main/java/com/ihomy/common/ReportAggregateUtil.java =====
package com.ihomy.common;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
/**
 * 报表聚合公共工具(V9.72):AI/天气用量报表共用的时间范围解析 + 时间桶零填充。
 * 消除 WeatherService 与 AiStatsService 里重复的 range switch 与零填充逻辑。
 */
public final class ReportAggregateUtil {
    private ReportAggregateUtil() {
    }
    /** 时间范围规格:start(含)、end(不含)、DB DATE_FORMAT 格式、Java 时间格式、步进单位 */
    public record RangeSpec(LocalDateTime start, LocalDateTime end, String fmt, String javaPatte
    rn, ChronoUnit unit) {
    }
    /** 解析时间范围:24h(默认)/month/30d/year */
    public static RangeSpec resolveRange(String range) {
        LocalDateTime now = LocalDateTime.now();
```

```
              :d="linePath"
              fill="none"
              stroke="url(#wtLineGrad)"
              stroke-width="2.5"
              stroke-linecap="round"
              stroke-linejoin="round"
            />
            <template v-for="(d, i) in dayMarks" :key="'dm' + i">
              <line :x1="d.x" :y1="PT - 8" :x2="d.x" :y2="CHART_H - PB" class="wc-day-sep" /
>
              <text :x="d.x + 5" :y="PT - 14" class="wc-daylabel">{{ d.label }}</text>
            </template>
            <template v-for="(lb, i) in extremeLabels" :key="'el' + i">
              <text :x="lb.x" :y="lb.y" class="wc-extreme" :fill="lb.fill">{{ lb.label }}</t
ext>
            </template>
            <template v-for="(lb, i) in timeLabels" :key="'xl' + i">
              <text :x="lb.x" :y="CHART_H - 10" class="wc-xlabel">{{ lb.label }}</text>
            </template>
            <g v-if="hoverIdx >= 0 && chartData">
              <line :x1="chartData.pts[hoverIdx].x" y1="32" :x2="chartData.pts[hoverIdx].x"
:y2="CHART_H - 36" class="wc-hover-line" />
              <circle :cx="chartData.pts[hoverIdx].x" :cy="chartData.pts[hoverIdx].y" r="5"
class="wc-hover-dot" />
            </g>
            <rect
              v-for="(p, i) in chartData?.pts || []"
              :key="'hr' + i"
              :x="p.x - hoverStep / 2"
              y="0"
              :width="hoverStep"
              :height="CHART_H"
              fill="transparent"
              @mouseenter="hoverIdx = i"
            />
          </svg>
          <div v-if="hoverIdx >= 0 && chartData" class="wc-tooltip" :style="tooltipStyle">
            <div class="wct-time">{{ fmtHour(chartData.pts[hoverIdx].h.fxTime) }}</div>
            <div class="wct-main">
              <i :class="'qi-' + chartData.pts[hoverIdx].h.icon" style="font-size: 16px"></i
>
              {{ chartData.pts[hoverIdx].t }}° {{ chartData.pts[hoverIdx].h.text }}
            </div>
            <div v-if="Number(chartData.pts[hoverIdx].h.pop) > 0" class="wct-sub">💧 {{ char
tData.pts[hoverIdx].h.pop }}%</div>
            <div class="wct-sub">
              {{ chartData.pts[hoverIdx].h.windDir }} {{ chartData.pts[hoverIdx].h.windScale
 }}{{ $t('weatherPage.level') }}
            </div>
          </div>
```

```
        </div>
      </div>
    </div>
    <!-- 10 天预报:前 3 天 7 列细列,后 7 天 3 列精简列(近细远粗) -->
    <div v-if="detail?.daily?.length" class="card">
      <div class="section-label">{{ $t('weatherPage.daily') }}</div>
      <el-table v-if="dailyMain.length" :data="dailyMain" size="small" stripe>
        <el-table-column :label="$t('weatherPage.date')" width="110">
          <template #default="{ row }">{{ fmtDate(row.fxDate) }}</template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.dayWeather')" min-width="140">
          <template #default="{ row }"> <i :class="'qi-' + row.iconDay" class="wt-td-icon"><
/i> {{ row.textDay }} </template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.nightWeather')" min-width="140">
          <template #default="{ row }"> <i :class="'qi-' + row.iconNight" class="wt-td-icon"
></i> {{ row.textNight }} </template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.tempRange')" min-width="220">
          <template #default="{ row }">
            <div class="wt-trange">
              <span class="wt-lo">{{ row.tempMin }}°</span>
              <div class="wt-tbar-track">
                <div class="wt-tbar-fill" :style="tbarStyle(row)"></div>
                <div class="wt-tbar-dot" :style="{ left: tbarDotLeft(row) }"></div>
              </div>
              <span class="wt-hi">{{ row.tempMax }}°</span>
            </div>
          </template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.uv')" width="70">
          <template #default="{ row }">{{ row.uvIndex }}</template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.wind')" width="120">
          <template #default="{ row }">{{ row.windDirDay }} {{ row.windScaleDay }}{{ $t('wea
therPage.level') }}</template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.sunSunset')" width="150">
          <template #default="{ row }">
            ☀ {{ (row.sunrise || '').slice(11, 16) }} / 🌙 {{ (row.sunset || '').slice(11, 1
6) }}
          </template>
        </el-table-column>
      </el-table>
      <el-table v-if="dailyRest.length" :data="dailyRest" size="small" stripe class="wt-dail
y-rest">
        <el-table-column :label="$t('weatherPage.date')" width="110">
          <template #default="{ row }">{{ fmtDate(row.fxDate) }}</template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.dayWeather')" min-width="140">
```

```
          <template #default="{ row }"> <i :class="'qi-' + row.iconDay" class="wt-td-icon"><
/i> {{ row.textDay }} </template>
        </el-table-column>
        <el-table-column :label="$t('weatherPage.tempRange')" min-width="220">
          <template #default="{ row }">
            <div class="wt-trange">
              <span class="wt-lo">{{ row.tempMin }}°</span>
              <div class="wt-tbar-track">
                <div class="wt-tbar-fill" :style="tbarStyle(row)"></div>
                <div class="wt-tbar-dot" :style="{ left: tbarDotLeft(row) }"></div>
              </div>
              <span class="wt-hi">{{ row.tempMax }}°</span>
            </div>
          </template>
        </el-table-column>
      </el-table>
    </div>
  </div>
  <!-- ── L3 空气与天空(安静块) ── -->
  <div class="wt-zone">
    <div class="wt-zone-label">{{ $t('weatherPage.zone.env') }}</div>
    <!-- 空气质量:常驻主指标,明细默认收起 -->
    <div v-if="detail?.air" class="card wt-card-quiet">
      <div class="section-label wt-section-head">
        <span>{{ $t('weatherPage.air') }}</span>
        <button class="wt-more" type="button" @click="airDetailOpen = !airDetailOpen">
          {{ airDetailOpen ? $t('weatherPage.airDetail.collapse') : $t('weatherPage.airDetai
l.expand') }}
          <span class="wt-more-arrow" :class="{ open: airDetailOpen }">▾</span>
        </button>
      </div>
      <div class="wt-air">
        <div class="wt-air-main">
          <span class="wa-aqi">{{ detail.air.aqi }}</span>
          <span class="wa-cat">{{ detail.air.category }}</span>
          <span v-if="detail.air.primary" class="wa-primary">{{ $t('weatherPage.primaryPollu
tant') }}: {{ detail.air.primary }}</span>
          <span class="wa-item"
            >PM2.5 <b>{{ detail.air.pm2p5 }}</b></span
          >
        </div>
        <div v-if="detail.air.adviceGeneral" class="wt-air-health">
          <span class="wah-label">{{ $t('weatherPage.healthAdvice') }}</span
          >{{ detail.air.adviceGeneral }}
        </div>
      </div>
      <div v-if="airDetailOpen" class="wt-air">
        <div class="wt-air-items">
          <span class="wa-item"
            >PM10 <b>{{ detail.air.pm10 }}</b></span
```

```
            >
            <span class="wa-item"
              >NO₂ <b>{{ detail.air.no2 }}</b></span
            >
            <span class="wa-item"
              >SO₂ <b>{{ detail.air.so2 }}</b></span
            >
            <span class="wa-item"
              >O₃ <b>{{ detail.air.o3 }}</b></span
            >
            <span class="wa-item"
              >CO <b>{{ detail.air.co }}</b></span
            >
          </div>
          <div v-if="detail.air.adviceSensitive" class="wt-air-health">
            <span class="wah-label">{{ $t('weatherPage.sensitiveAdvice') }}</span
            >{{ detail.air.adviceSensitive }}
          </div>
          <!-- 24 小时空气质量走势 -->
          <div v-if="airTrend" class="wt-air-trend">
            <div class="wat-head">
              <span class="wat-title">{{ $t('weatherPage.airHourly') }}</span>
              <span class="wat-range">AQI {{ airTrend.min }} ~ {{ airTrend.max }}</span>
            </div>
            <div class="wat-chart-wrap" @mouseleave="airHover = -1">
              <svg :viewBox="`0 0 ${AIR_W} ${AIR_H}`" class="wat-chart">
                <path :d="airArea" :fill="airPeakColor" opacity="0.12" />
                <path :d="airLine" fill="none" :stroke="airPeakColor" stroke-width="2" stroke-
linejoin="round" stroke-linecap="round" />
                <template v-for="(lb, i) in airTrend.labels" :key="'al' + i">
                  <text :x="lb.x" :y="AIR_H - 8" class="wc-xlabel">{{ lb.label }}</text>
                </template>
                <g v-if="airHover >= 0">
                  <line
                    :x1="airTrend.pts[airHover].x"
                    :y1="AIR_PT - 8"
                    :x2="airTrend.pts[airHover].x"
                    :y2="AIR_H - AIR_PB"
                    class="wc-hover-line"
                  />
                  <circle
                    :cx="airTrend.pts[airHover].x"
                    :cy="airTrend.pts[airHover].y"
                    r="4"
                    :fill="aqiColor(airTrend.pts[airHover].h.level)"
                    class="wat-dot"
                  />
                </g>
                <rect
                  v-for="(p, i) in airTrend.pts"
```

```
                :key="'ar' + i"
                :x="p.x - 15"
                y="0"
                width="30"
                :height="AIR_H"
                fill="transparent"
                @mouseenter="airHover = i"
              />
            </svg>
            <div v-if="airHover >= 0" class="wc-tooltip" :style="airTooltipStyle">
              <div class="wct-time">{{ fmtHour(airTrend.pts[airHover].h.fxTime) }}</div>
              <div class="wct-main" :style="{ color: aqiColor(airTrend.pts[airHover].h.level
) }">
                AQI {{ airTrend.pts[airHover].v }} · {{ airTrend.pts[airHover].h.category }}
              </div>
              <div v-if="airTrend.pts[airHover].h.primary" class="wct-sub">
                {{ $t('weatherPage.primaryPollutant') }}: {{ airTrend.pts[airHover].h.primar
y }}
              </div>
            </div>
          </div>
        </div>
        <!-- 未来 3 天空气质量 -->
        <div v-if="detail.airDaily?.length" class="wt-air-days">
          <div class="wat-head">
            <span class="wat-title">{{ $t('weatherPage.airDaily') }}</span>
          </div>
          <div class="wad-list">
            <div v-for="d in detail.airDaily" :key="d.fxDate" class="wad-item" :style="{ bor
derLeftColor: aqiColor(d.level) }">
              <span class="wad-date">{{ fmtMD(d.fxDate) }}</span>
              <span class="wad-aqi" :style="{ color: aqiColor(d.level) }">{{ d.aqi }}</span>
              <span class="wad-cat" :style="{ background: aqiColor(d.level) + '22', color: a
qiColor(d.level) }">{{ d.category }}</span>
              <span v-if="d.primary" class="wad-primary">{{ d.primary }}</span>
            </div>
          </div>
        </div>
      </div>
    </div>
    <!-- 日月与晨昏:世界地图 + 晨昏线,点击地点看此刻阶段(sun-info 拿不到时整块不渲染) -->
    <TerminatorMap
      v-if="astro"
      :astro="astro"
      :city="astro.city || ''"
      :lat="astro.lat"
      :lng="astro.lng"
      :subsolar="astro.subsolar"
      :generated-at="astro.generatedAt"
    />
```

```
        <!-- 生活指数:紧凑网格,点开看全文 -->
        <div v-if="detail?.indices?.length" class="card wt-card-quiet">
          <div class="section-label">{{ $t('weatherPage.indices') }}</div>
          <div class="wt-indices">
            <button v-for="(ix, i) in detail.indices" :key="i" class="wt-index" type="button" @c
    lick="openIndex(ix)">
              <span class="wi-head">
                <span class="wi-name">{{ ix.name }}</span
                ><span class="wi-cat">{{ ix.category }}</span>
              </span>
              <span class="wi-text">{{ ix.text }}</span>
            </button>
          </div>
        </div>
      </div>
      <el-empty v-if="!loading && !detail" :description="$t('weatherPage.empty')" />
    </div>
    <!-- 预警详情:append-to-body 免得被左侧导航盖住;限宽避免长正文撑满屏 -->
    <el-dialog v-model="warnDialog" :title="activeWarn?.typeName" append-to-body width="min(640p
    x, 92vw)">
      <div v-if="activeWarn" class="wt-warn-dialog">
        <div class="wt-warn-head">
          <span class="wt-warn-title" :style="{ color: warnLevelColor(activeWarn.level) }">
            {{ activeWarn.level }}{{ $t('weatherPage.warningSuffix') }}
          </span>
          <span v-if="activeWarn.senderName" class="wt-warn-sender">{{ activeWarn.senderName }}<
    /span>
          <span class="wt-warn-time">{{ fmtDateTime(activeWarn.startTime) }} ~ {{ fmtDateTime(ac
    tiveWarn.endTime) }}</span>
        </div>
        <div class="wt-warn-text">{{ activeWarn.text }}</div>
        <div v-if="activeWarn.instruction" class="wt-warn-instruction">
          {{ $t('weatherPage.defenseGuide') }}: {{ activeWarn.instruction }}
        </div>
      </div>
    </el-dialog>
    <!-- 生活指数全文 -->
    <el-dialog v-model="indexDialog" :title="activeIndex?.name" append-to-body width="min(560px,
     92vw)">
      <div v-if="activeIndex" class="wt-index-dialog">
        <div class="wi-cat">{{ activeIndex.category }}</div>
        <div class="wi-text">{{ activeIndex.text }}</div>
      </div>
    </el-dialog>
  </div>
</template>
<script setup>
import { computed, nextTick, onBeforeUnmount, onMounted, ref, inject, watch } from 'vue'
import { useI18n } from 'vue-i18n'
import Breadcrumb from '@/components/Breadcrumb.vue'
```

```
import TerminatorMap from '@/views/weather/TerminatorMap.vue'
import { aqiColor, warnLevelColor, WARN_LEVEL_ORDER } from '@/utils/dict'
import { SUN_LIGHT_KEY } from '@/utils/useSunLight'
import { publicApi } from '@/api'
const { t } = useI18n()
const sunLight = inject(SUN_LIGHT_KEY, null)
const loading = ref(false)
const detail = ref(null)
const warnings = computed(() => detail.value?.warning || [])
// 预警按级别降序:最严重的排在最右,其余向左延伸
const sortedWarnings = computed(() =>
  [...warnings.value].sort((a, b) => WARN_LEVEL_ORDER.indexOf(b.level) - WARN_LEVEL_ORDER.indexO
    f(a.level)),
)
const warnTint = (level) => warnLevelColor(level) + '14'
const todayHigh = computed(() => detail.value?.daily?.[0]?.tempMax ?? '—')
const todayLow = computed(() => detail.value?.daily?.[0]?.tempMin ?? '—')
const minutelySummary = computed(() => detail.value?.minutely?.summary || '')
// 今日日期标签:跨零点自动刷新(每分钟检查一次)
const fmtToday = () => {
  const d = new Date()
  return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).
    padStart(2, '0')}`
}
const todayLabel = ref(fmtToday())
const todayTimer = setInterval(() => {
  todayLabel.value = fmtToday()
}, 60000)
onBeforeUnmount(() => clearInterval(todayTimer))
const nf = (key) => detail.value?.nowFull?.[key] ?? '—'
const fmtHour = (s) => (s || '').slice(11, 16)
const fmtDate = (s) => {
  if (!s) return ''
  const d = new Date(s)
  const weekdays = [
    t('weatherPage.w0'),
    t('weatherPage.w1'),
    t('weatherPage.w2'),
    t('weatherPage.w3'),
    t('weatherPage.w4'),
    t('weatherPage.w5'),
    t('weatherPage.w6'),
  ]
  return `${d.getMonth() + 1}/${d.getDate()} ${weekdays[d.getDay()]}`
}
const fmtDateTime = (s) => (s || '').replace('T', ' ').slice(5, 16)
// "2026-09-25" / ISO 串 → "9/25"(按字符串取,避免 new Date 在非 +08:00 环境偏移一天)
const fmtMD = (s) => {
  const m = /^(\d{4})-(\d{2})-(\d{2})/.exec(s || '')
  return m ? `${Number(m[2])}/${Number(m[3])}` : ''
```

```
}
// ── 气象预警(实况右侧,右起排列 + 详情弹窗) ──
const warnRowEl = ref(null)
const warnDialog = ref(false)
const activeWarn = ref(null)
const openWarning = (w) => {
  activeWarn.value = w
  warnDialog.value = true
}
// row-reverse 下初始滚动位置各浏览器不一致:按第一条(最右)chip 的实际位置纠正
function scrollWarnRow() {
  const el = warnRowEl.value
  const first = el?.firstElementChild
  if (!el || !first || el.scrollWidth <= el.clientWidth) return
  el.scrollLeft += first.getBoundingClientRect().right - el.getBoundingClientRect().right
}
watch(sortedWarnings, () => nextTick(scrollWarnRow))
onMounted(() => nextTick(scrollWarnRow))
// ── 生活指数(紧凑网格 + 全文弹窗) ──
const indexDialog = ref(false)
const activeIndex = ref(null)
const openIndex = (ix) => {
  activeIndex.value = ix
  indexDialog.value = true
}
// ── 空气质量明细折叠 ──
const airDetailOpen = ref(false)
// ── 视图切换(折线/卡片 + 24h/72h,localStorage 记忆) ──
const hourlyView = ref(localStorage.getItem('ihomy:hourly-view') || 'chart')
const setHourlyView = (v) => {
  hourlyView.value = v
  localStorage.setItem('ihomy:hourly-view', v)
}
// 72h 为逐 2 小时(后端每小时采样一点);数据缺失(凭证不支持 hours 参数)时回落 24h,避免空图
const hourlyRange = ref(localStorage.getItem('ihomy:hourly-range') === '72h' ? '72h' : '24h')
const setHourlyRange = (r) => {
  hourlyRange.value = r
  localStorage.setItem('ihomy:hourly-range', r)
}
const hourly72 = computed(() => detail.value?.hourly72 || [])
const effRange = computed(() => (hourlyRange.value === '72h' && hourly72.value.length ? '72h' :
    '24h'))
const hourPoints = computed(() => (effRange.value === '72h' ? hourly72.value : detail.value?.hou
    rly || []))
const isDayStart = (i) =>
  i === 0 || (hourPoints.value[i]?.fxTime || '').slice(0, 10) !== (hourPoints.value[i - 1]?.fxTi
    me || '').slice(0, 10)
// ── 温度 → 颜色(7 段渐变,-20°冷蓝→42°暖红) ──
const TEMP_STOPS = [
  [-20, 74, 111, 212],
```

```
  [-5, 95, 159, 224],
  [5, 98, 184, 201],
  [15, 143, 201, 143],
  [22, 224, 201, 95],
  [28, 224, 155, 95],
  [34, 224, 120, 69],
  [42, 216, 74, 58],
]
const tempColor = (t) => {
  const s = TEMP_STOPS
  if (t <= s[0][0]) return `rgb(${s[0][1]},${s[0][2]},${s[0][3]})`
  for (let i = 0; i < s.length - 1; i++) {
    if (t <= s[i + 1][0]) {
      const f = (t - s[i][0]) / (s[i + 1][0] - s[i][0])
      return `rgb(${Math.round(s[i][1] + (s[i + 1][1] - s[i][1]) * f)},${Math.round(s[i][2] + (s
    [i + 1][2] - s[i][2]) * f)},${Math.round(s[i][3] + (s[i + 1][3] - s[i][3]) * f)})`
    }
  }
  const l = s[s.length - 1]
  return `rgb(${l[1]},${l[2]},${l[3]})`
}
// ── 折线图几何(宽度随点数变化:72h 点位固定 46px 后横向滚动) ──
const CHART_W = 720,
  CHART_H = 230,
  PT = 34,
  PB = 40,
  PL = 14,
  PR = 14
const hoverIdx = ref(-1)
const chartW = computed(() => (effRange.value === '72h' ? Math.max(CHART_W, hourPoints.value.len
    gth * 46) : CHART_W))
const chartData = computed(() => {
  const hs = hourPoints.value
  if (!hs.length) return null
  const temps = hs.map((h) => Number(h.temp) || 0)
  const tMin = Math.min(...temps) - 1,
    tMax = Math.max(...temps) + 1
  const n = hs.length
  const xOf = (i) => PL + (i / (n - 1)) * (chartW.value - PL - PR)
  const yOf = (t) => PB + (1 - (t - tMin) / (tMax - tMin || 1)) * (CHART_H - PB - PT)
  const pts = hs.map((h, i) => ({ x: xOf(i), y: yOf(Number(h.temp) || 0), t: Number(h.temp) || 0
    , h }))
  return { pts, tMin, tMax, temps }
})
// 天数分隔线(72h 跨三天,标出每天的起点)
const dayMarks = computed(() => {
  if (effRange.value !== '72h' || !chartData.value) return []
  return chartData.value.pts
    .map((p, i) => ({ p, i }))
    .filter(({ i }) => i > 0 && isDayStart(i))
```

```
    .map(({ p, i }) => ({ x: p.x, label: fmtMD(hourPoints.value[i].fxTime) }))
})
const hoverStep = computed(() => {
  const n = chartData.value?.pts.length || 1
  return (chartW.value - PL - PR) / (n - 1)
})
// Catmull-Rom → 平滑 Bezier
const catmullRom = (pts) => {
  if (pts.length < 2) return ''
  const tension = 0.35
  let d = `M${pts[0].x},${pts[0].y}`
  for (let i = 0; i < pts.length - 1; i++) {
    const p0 = pts[Math.max(i - 1, 0)]
    const p1 = pts[i]
    const p2 = pts[i + 1]
    const p3 = pts[Math.min(i + 2, pts.length - 1)]
    const cp1x = p1.x + (p2.x - p0.x) * tension
    const cp1y = p1.y + (p2.y - p1.y) * tension
    const cp2x = p2.x - (p3.x - p1.x) * tension
    const cp2y = p2.y - (p3.y - p2.y) * tension
    d += `C${cp1x},${cp1y} ${cp2x},${cp2y} ${p2.x},${p2.y}`
  }
  return d
}
const linePath = computed(() => (chartData.value ? catmullRom(chartData.value.pts) : ''))
const areaPath = computed(() => {
  if (!chartData.value) return ''
  const { pts } = chartData.value
  return catmullRom(pts) + `L${pts[pts.length - 1].x},${CHART_H - PB}L${pts[0].x},${CHART_H - PB
    }Z`
})
// 最高/最低标注
const extremeLabels = computed(() => {
  if (!chartData.value) return []
  const { pts } = chartData.value
  const maxI = pts.reduce((m, p, i) => (p.t > pts[m].t ? i : m), 0)
  const minI = pts.reduce((m, p, i) => (p.t < pts[m].t ? i : m), 0)
  const out = [{ x: pts[maxI].x, y: pts[maxI].y - 10, label: pts[maxI].t + '°', fill: '#c07a4a'
    }]
  if (minI !== maxI) out.push({ x: pts[minI].x, y: pts[minI].y + 16, label: pts[minI].t + '°', f
    ill: '#6a8ab0' })
  return out
})
// 底部时间标签(每 3 小时)
const timeLabels = computed(() => {
  if (!chartData.value) return []
  return chartData.value.pts.filter((_, i) => i % 3 === 0).map((p) => ({ x: p.x, label: fmtHour(
    p.h.fxTime) }))
})
// 悬浮 tooltip 定位(百分比相对 .wt-chart-inner,内层宽度即 chartW,横向滚动时仍跟随)
```

```
const tooltipStyle = computed(() => {
  if (hoverIdx.value < 0 || !chartData.value) return { display: 'none' }
  const p = chartData.value.pts[hoverIdx.value]
  const left = Math.min(Math.max(p.x / chartW.value, 0.08), 0.92) * 100
  const top = (p.y / CHART_H) * 100
  return { left: left + '%', top: top + '%' }
})
// ── 日月天文(纯天文计算:取自 /public/sun-info,后端 SolarUtil 用 NOAA/Meeus 算法算,不依赖天气 A
    PI;
//    故天气凭证未配置时晨昏地图照常显示。渲染交给 <TerminatorMap>) ──
const astro = ref(null)
const astroLoading = ref(false)
// 晨昏地图依赖后端新增的 subsolar 直射点;老后端/应用启动时取的旧数据没有该字段 → 当日则重取一次
const astroStale = () => !!astro.value && astro.value.date === todayLabel.value && !astro.value.
    subsolar && !astroLoading.value
// ── 空气质量 24 小时走势(和风 airquality/v1/hourly) ──
const AIR_W = 720,
  AIR_H = 150,
  AIR_PT = 20,
  AIR_PB = 26,
  AIR_PL = 16,
  AIR_PR = 16
const airHover = ref(-1)
const airTrend = computed(() => {
  const hs = detail.value?.airHourly || []
  if (hs.length < 2) return null
  const vals = hs.map((h) => Number(h.aqi) || 0)
  const min = Math.min(...vals),
    max = Math.max(...vals)
  const lo = Math.max(0, min - 10),
    hi = max + 10
  const n = hs.length
  const xOf = (i) => AIR_PL + (i / (n - 1)) * (AIR_W - AIR_PL - AIR_PR)
  const yOf = (v) => AIR_PB + (1 - (v - lo) / (hi - lo || 1)) * (AIR_H - AIR_PB - AIR_PT)
  const pts = hs.map((h, i) => ({ x: xOf(i), y: yOf(Number(h.aqi) || 0), v: Number(h.aqi) || 0,
    h }))
  return {
    pts,
    min,
    max,
    peakLevel: Math.max(...hs.map((h) => Number(h.level) || 1)),
    labels: pts.filter((_, i) => i % 6 === 0).map((p) => ({ x: p.x, label: fmtHour(p.h.fxTime) }
    )),
  }
})
const airPeakColor = computed(() => aqiColor(airTrend.value?.peakLevel || 1))
const airLine = computed(() => (airTrend.value ? airTrend.value.pts.map((p, i) => `${i ? 'L' : '
    M'}${p.x},${p.y}`).join('') : ''))
const airArea = computed(() => {
  const tr = airTrend.value
```

```
  if (!tr) return ''
  const pts = tr.pts
  return `${airLine.value}L${pts[pts.length - 1].x},${AIR_H - AIR_PB}L${pts[0].x},${AIR_H - AIR_
    PB}Z`
})
const airTooltipStyle = computed(() => {
  if (airHover.value < 0 || !airTrend.value) return { display: 'none' }
  const p = airTrend.value.pts[airHover.value]
  return {
    left: Math.min(Math.max(p.x / AIR_W, 0.08), 0.92) * 100 + '%',
    top: (p.y / AIR_H) * 100 + '%',
  }
})
// ── N 天温度区间渐变横条 ──
const dailyRange = computed(() => {
  const d = detail.value?.daily || []
  if (!d.length) return null
  const min = Math.min(...d.map((x) => Number(x.tempMin)))
  const max = Math.max(...d.map((x) => Number(x.tempMax)))
  return { min, max, range: max - min || 1 }
})
const tbarStyle = (row) => {
  const r = dailyRange.value
  if (!r) return { background: '#ccc' }
  const lo = Number(row.tempMin),
    hi = Number(row.tempMax)
  return {
    left: ((lo - r.min) / r.range) * 100 + '%',
    width: (Math.max(hi - lo, 2) / r.range) * 100 + '%',
    background: `linear-gradient(90deg, ${tempColor(lo)}, ${tempColor(hi)})`,
  }
}
const tbarDotLeft = (row) => {
  const r = dailyRange.value
  if (!r) return '50%'
  const avg = (Number(row.tempMin) + Number(row.tempMax)) / 2
  return ((avg - r.min) / r.range) * 100 + '%'
}
// 近细远粗:前 3 天 7 列细列,后 7 天 3 列精简列(拆两张表,EP 列是全局的没法按行隐藏)
const dailyMain = computed(() => (detail.value?.daily || []).slice(0, 3))
const dailyRest = computed(() => (detail.value?.daily || []).slice(3))
const load = async () => {
  if (sunLight?.weatherDetail?.value) detail.value = sunLight.weatherDetail.value
  // 光影层应用启动时已取过当日 sun-info(同一份数据),先拿来立即渲染省一次请求;
  // 但光线测试台可能按别的日期取过,日期不符就自己再取一次,免得卡片显示的是另一天的日月时刻
  const shared = sunLight?.sunInfo?.value
  if (shared?.date === todayLabel.value) astro.value = shared
  loading.value = true
  // 天气详情与日月天文并行:后者纯天文计算(不依赖天气凭证),故天气凭证未配/接口失败也不影响晨昏区
    块
```

```
  const [data, sun] = await Promise.all([
    publicApi.getWeatherDetail().catch(() => null),
    astroStale() || astro.value ? Promise.resolve(null) : publicApi.getSunInfo().catch(() => nul
    l),
  ])
  if (data) {
    detail.value = data
    if (sunLight?.weatherDetail) sunLight.weatherDetail.value = data
  }
  if (sun) astro.value = sun
  loading.value = false
  // 共享的那份缺 subsolar(后端未升级/启动时取于旧版本)→ 补一次取当日完整数据
  if (astroStale()) {
    astroLoading.value = true
    const fresh = await publicApi.getSunInfo().catch(() => null)
    astroLoading.value = false
    if (fresh) astro.value = fresh
  }
}
onMounted(load)
</script>
<style scoped>
.weather-page-body {
  display: flex;
  flex-direction: column;
}
/* 三个信息组:现在 / 未来 / 空气与天空,组内紧凑、组间留白 */
.wt-zone {
  display: flex;
  flex-direction: column;
  gap: 10px;
}
.wt-zone + .wt-zone {
  margin-top: 14px;
}
.wt-zone-label {
  font-size: 12px;
  letter-spacing: 0.5px;
  color: var(--color-text-tertiary);
}
/* L3 安静块:去掉阴影与悬浮位移,标题收小,靠内层 --color-card-2 分层 */
.wt-card-quiet {
  box-shadow: none;
  padding: 16px 20px;
}
.wt-card-quiet:hover {
  box-shadow: none;
  transform: none;
}
.wt-card-quiet .section-label {
```

```
  font-size: 12.5px;
  color: var(--color-text-secondary);
  margin-bottom: 12px;
}
.wt-card-quiet .section-label:hover {
  color: var(--color-text-secondary);
}
/* ── L1 实况 ── */
.wt-hero {
  padding: 20px;
}
.wt-hero-top {
  display: flex;
  align-items: flex-start;
  gap: 16px;
}
.wt-now {
  flex: 1 1 auto;
  min-width: 0;
}
.wt-city {
  font-size: 14px;
  color: var(--color-text-secondary);
}
.wt-main {
  display: flex;
  align-items: center;
  gap: 10px;
  margin: 8px 0 4px;
  flex-wrap: wrap;
}
.wt-big-icon {
  font-size: 56px;
  line-height: 1;
}
.wt-temp {
  font-size: 56px;
  font-weight: 700;
  line-height: 1;
  font-variant-numeric: tabular-nums;
}
.wt-degree {
  font-size: 28px;
  opacity: 0.5;
}
.wt-hilo {
  display: flex;
  flex-direction: column;
  gap: 2px;
}
```

```
.wt-hi {
  color: #c07a4a;
  font-weight: 600;
  font-size: 14px;
}
.wt-lo {
  color: #6a8ab0;
  font-weight: 600;
  font-size: 14px;
}
.wt-text {
  font-size: 16px;
  color: var(--color-text-secondary);
}
.wt-metrics {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
  gap: 10px;
  margin-top: 16px;
}
.wt-metric {
  background: var(--color-card-2);
  border-radius: 10px;
  padding: 10px 14px;
  display: flex;
  flex-direction: column;
  gap: 4px;
}
.wm-label {
  font-size: 12px;
  color: var(--color-text-secondary);
}
.wm-val {
  font-size: 15px;
  font-weight: 600;
  font-variant-numeric: tabular-nums;
}
/* 预警列:右起排列,最严重在最右;溢出时横向滚动并初始停在最右 */
.wt-warnings {
  flex: 0 1 46%;
  display: flex;
  flex-direction: row-reverse;
  justify-content: flex-start;
  gap: 10px;
  overflow-x: auto;
  padding-bottom: 2px;
}
.wt-warn-chip {
  flex: 0 0 280px;
  border: 0;
```

```
  border-left: 3px solid;
  border-radius: 10px;
  padding: 10px 12px;
  text-align: left;
  cursor: pointer;
  display: flex;
  flex-direction: column;
  gap: 4px;
  min-width: 0;
}
.wt-warn-chip:focus-visible {
  outline: 2px solid var(--color-brand);
  outline-offset: 2px;
}
.wt-warn-title {
  font-weight: 700;
  font-size: 13px;
}
.wt-warn-time {
  font-size: 11px;
  color: var(--color-text-secondary);
}
.wt-warn-text {
  font-size: 12px;
  line-height: 1.6;
  display: -webkit-box;
  -webkit-line-clamp: 3;
  line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
.wt-warn-dialog {
  display: flex;
  flex-direction: column;
  gap: 8px;
}
.wt-warn-head {
  display: flex;
  align-items: baseline;
  gap: 10px;
  flex-wrap: wrap;
}
.wt-warn-sender {
  font-size: 12px;
  color: var(--color-text-secondary);
}
.wt-warn-instruction {
  font-size: 12px;
  line-height: 1.6;
  padding: 8px 10px;
```

```
  background: rgba(var(--color-brand-rgb), 0.08);
  border-radius: 8px;
  color: var(--color-text-secondary);
  white-space: pre-line;
}
/* ── 控件 ── */
.wt-section-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}
.wt-toggles {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}
.wt-vt-text {
  width: auto;
  padding: 0 10px;
  font-size: 12px;
}
.wt-more {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  border: 0;
  background: transparent;
  cursor: pointer;
  font-size: 12px;
  color: var(--color-text-secondary);
  padding: 0;
}
.wt-more:hover {
  color: var(--color-text);
}
.wt-more-arrow {
  display: inline-block;
  transition: transform 0.2s;
}
.wt-more-arrow.open {
  transform: rotate(180deg);
}
/* 分钟降水一句话(并入逐小时卡顶部) */
.wt-minutely-banner {
  font-size: 12px;
  line-height: 1.6;
  background: var(--color-card-2);
  color: var(--color-text-secondary);
```

```
  border-radius: 8px;
  padding: 8px 12px;
  margin-bottom: 10px;
}
/* ── 逐小时 ── */
.wt-hourly {
  display: flex;
  gap: 8px;
  overflow-x: auto;
  padding-bottom: 6px;
}
.wt-hour {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
  background: var(--color-card-2);
  border-radius: 10px;
  padding: 10px 8px;
  min-width: 76px;
  flex: none;
}
.wh-time {
  font-size: 12px;
  color: var(--color-text-secondary);
  font-variant-numeric: tabular-nums;
}
.wh-icon {
  font-size: 26px;
}
.wh-temp {
  font-size: 15px;
  font-weight: 700;
}
.wh-text {
  font-size: 11px;
  color: var(--color-text-secondary);
}
.wh-pop {
  font-size: 11px;
  color: #6a8ab0;
}
.wh-wind {
  font-size: 11px;
  color: var(--color-text-secondary);
}
.wt-chart-scroll {
  overflow-x: auto;
  overflow-y: hidden;
  padding-bottom: 4px;
```

```
}
.wt-chart-inner {
  position: relative;
}
.wt-chart-inner .wt-chart {
  max-width: none;
}
.wc-xlabel {
  /* 10px 次要色在暖色纸底上几乎看不清,轴标签是读数不是装饰,提到正文色并放大一号 */
  font-size: 11px;
  fill: var(--color-text);
  text-anchor: middle;
  font-variant-numeric: tabular-nums;
}
.wc-extreme {
  font-size: 11px;
  font-weight: 700;
  text-anchor: middle;
}
.wc-hover-line {
  stroke: var(--color-text-secondary);
  stroke-dasharray: 3 3;
  opacity: 0.4;
}
.wc-hover-dot {
  fill: var(--color-brand);
  stroke: #fffdf8;
  stroke-width: 2;
}
.wc-tooltip {
  position: absolute;
  transform: translate(-50%, -120%);
  background: var(--color-card-2);
  border: 1px solid var(--color-border);
  border-radius: 10px;
  padding: 8px 12px;
  font-size: 12px;
  pointer-events: none;
  box-shadow: 0 3px 12px rgba(0, 0, 0, 0.08);
  white-space: nowrap;
  z-index: 5;
}
.wct-time {
  color: var(--color-text-secondary);
  font-size: 11px;
}
.wct-main {
  font-weight: 700;
  font-size: 13px;
  display: flex;
```

```
  align-items: center;
  gap: 4px;
}
.wct-sub {
  color: var(--color-text-secondary);
  font-size: 11px;
}
/* 72h 卡片模式按天插分隔 */
.wt-hour-sep {
  flex: none;
  align-self: stretch;
  display: flex;
  align-items: center;
  padding: 0 6px;
  font-size: 12px;
  font-weight: 600;
  color: var(--color-text-secondary);
  border-left: 1px dashed var(--color-border);
}
.wc-day-sep {
  stroke: var(--color-border);
  stroke-dasharray: 4 4;
}
.wc-daylabel {
  font-size: 10px;
  font-weight: 600;
  fill: var(--color-text-secondary);
}
/* ── N 天温度横条 ── */
.wt-daily-rest {
  margin-top: 8px;
}
.wt-trange {
  display: flex;
  align-items: center;
  gap: 8px;
}
.wt-tbar-track {
  flex: 1;
  height: 8px;
  background: rgba(58, 46, 34, 0.1);
  border-radius: 4px;
  position: relative;
  min-width: 90px;
}
.wt-tbar-fill {
  position: absolute;
  top: 0;
  height: 100%;
  border-radius: 4px;
```

```
}
.wt-tbar-dot {
  position: absolute;
  top: 50%;
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background: rgba(58, 46, 34, 0.55);
  transform: translate(-50%, -50%);
}
.wt-td-icon {
  font-size: 18px;
}
/* ── 空气质量 ── */
.wt-air {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
.wt-air-main {
  display: flex;
  align-items: baseline;
  gap: 12px;
  flex-wrap: wrap;
}
.wa-aqi {
  font-size: 34px;
  font-weight: 700;
  line-height: 1;
}
.wa-cat {
  font-size: 15px;
  font-weight: 600;
}
.wa-primary {
  font-size: 12px;
  color: var(--color-text-secondary);
}
.wt-air-items {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}
.wa-item {
  background: var(--color-card-2);
  border-radius: 8px;
  padding: 6px 12px;
  font-size: 12px;
  color: var(--color-text-secondary);
}
```

```
.wa-item b {
  color: var(--color-text);
  margin-left: 4px;
  font-variant-numeric: tabular-nums;
}
.wt-air-health {
  font-size: 12px;
  line-height: 1.7;
  color: var(--color-text-secondary);
  background: rgba(107, 155, 107, 0.08);
  border-radius: 8px;
  padding: 10px 12px;
}
.wah-label {
  font-weight: 600;
  color: var(--color-text);
  margin-right: 6px;
}
.wt-air-trend,
.wt-air-days {
  display: flex;
  flex-direction: column;
  gap: 6px;
}
.wat-head {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 10px;
}
.wat-title {
  font-size: 13px;
  font-weight: 600;
}
.wat-range {
  font-size: 12px;
  color: var(--color-text-secondary);
  font-variant-numeric: tabular-nums;
}
.wat-chart-wrap {
  position: relative;
}
.wat-chart {
  width: 100%;
  height: auto;
  display: block;
}
.wat-dot {
  stroke: var(--color-card);
  stroke-width: 2;
```

```
}
.wad-list {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}
.wad-item {
  flex: 1 1 150px;
  display: flex;
  align-items: center;
  gap: 8px;
  background: var(--color-card-2);
  border-left: 3px solid;
  border-radius: 8px;
  padding: 8px 12px;
  font-size: 12px;
}
.wad-date {
  color: var(--color-text-secondary);
  font-variant-numeric: tabular-nums;
}
.wad-aqi {
  font-size: 17px;
  font-weight: 700;
  font-variant-numeric: tabular-nums;
}
.wad-cat {
  border-radius: 6px;
  padding: 2px 8px;
  font-weight: 600;
}
.wad-primary {
  margin-left: auto;
  color: var(--color-text-secondary);
}
/* ── 生活指数(紧凑网格) ── */
.wt-indices {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 10px;
}
.wt-index {
  background: var(--color-card-2);
  border: 0;
  border-radius: 10px;
  padding: 12px 14px;
  text-align: left;
  cursor: pointer;
  display: flex;
  flex-direction: column;
```

```
  gap: 6px;
  min-width: 0;
}
.wt-index:hover {
  background: var(--color-bg-2);
}
.wt-index:focus-visible {
  outline: 2px solid var(--color-brand);
  outline-offset: 2px;
}
.wi-head {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 8px;
}
.wi-name {
  font-size: 13px;
  font-weight: 600;
}
.wi-cat {
  font-size: 12px;
  color: var(--color-brand);
  font-weight: 600;
}
.wi-text {
  font-size: 12px;
  line-height: 1.6;
  color: var(--color-text-secondary);
  display: -webkit-box;
  -webkit-line-clamp: 3;
  line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
.wt-index-dialog {
  display: flex;
  flex-direction: column;
  gap: 10px;
}
.wt-index-dialog .wi-text {
  -webkit-line-clamp: unset;
  line-clamp: unset;
  display: block;
  overflow: visible;
}
/* 深色模式横条轨道 */
:global(html.dark) .wt-tbar-track {
  background: rgba(232, 220, 200, 0.12);
}
```

```
:global(html.dark) .wt-tbar-dot {
  background: rgba(232, 220, 200, 0.55);
}
:global(html.dark) .wc-hover-dot {
  stroke: var(--color-card);
}
@media (max-width: 900px) {
  .wt-hero-top {
    flex-direction: column;
  }
  .wt-warnings {
    width: 100%;
    flex: none;
    flex-direction: row;
  }
  .wt-warn-chip {
    flex: 0 0 78vw;
  }
}
@media (max-width: 768px) {
  .wt-metrics {
    grid-template-columns: repeat(2, 1fr);
  }
  .wt-big-icon {
    font-size: 44px;
  }
  .wt-temp {
    font-size: 44px;
  }
  .wt-toggles {
    gap: 8px;
  }
  .wad-item {
    flex: 1 1 100%;
  }
}
</style>
<!-- ===== frontend/src/views/wish/Wish.vue ===== -->
<!-- 愿望单页:家庭共享愿望,可标记实现/放弃,附分类标签与提出人 -->
<template>
  <div class="page">
    <Breadcrumb :items="[{ label: $t('wish.title') }]" />
    <PageToolbar>
      <div class="tb-right">
        <el-button v-if="userStore.isLoggedIn" type="primary" @click="openEditor()">{{ $t('wish.
    add') }}</el-button>
      </div>
    </PageToolbar>
    <el-tabs v-model="filter">
      <el-tab-pane :label="$t('more.all')" name="all" />
```

```
  <el-tab-pane :label="$t('wish.pending')" name="open" />
  <el-tab-pane :label="$t('wish.achieved')" name="done" />
</el-tabs>
<div v-loading="loading">
  <div v-if="filtered.length" class="wish-grid">
    <div
      v-for="w in filtered"
      :key="w.id"
      class="wish-card card"
      :class="'st' + (w.status === 'ACHIEVED' ? 1 : w.status === 'ABANDONED' ? 2 : 0)"
    >
      <div class="wish-top">
        <span class="wish-title">{{ w.title }}</span>
        <el-tag :type="statusTag(w.status)" size="small">{{ statusText(w.status) }}</el-tag>
      </div>
      <div v-if="w.reason" class="wish-reason">{{ w.reason }}</div>
      <div v-if="w.category" class="wish-cats">
        <el-tag v-for="c in w.category.split(',')" :key="c" size="small" effect="plain">{{ c
 }}</el-tag>
      </div>
      <div class="wish-meta">
        <span>{{ $t('wish.requested', { name: w.requesterName }) }}</span>
        <span v-if="w.status === 'ACHIEVED' && w.achievedAt">{{ $t('wish.achievedAt', { date
: w.achievedAt.slice(0, 10) }) }}</span>
        <span v-else>{{ w.createdAt.slice(0, 10) }}</span>
      </div>
      <div v-if="userStore.isLoggedIn" class="wish-actions">
        <el-button v-if="w.status === 'PENDING'" size="small" type="success" plain @click="o
nSetStatus(w, 'ACHIEVED')">{{
          $t('wish.achieved')
        }}</el-button>
        <el-button v-if="w.status === 'PENDING'" size="small" type="info" plain @click="onSe
tStatus(w, 'ABANDONED')">{{
          $t('wish.abandon')
        }}</el-button>
        <el-button v-if="w.status !== 'PENDING'" size="small" @click="onSetStatus(w, 'PENDIN
G')">{{ $t('wish.restore') }}</el-button>
        <el-tooltip :content="$t('common.edit')" placement="top" :show-after="300">
          <el-button size="small" text @click="openEditor(w)"
            ><el-icon><Edit /></el-icon
          ></el-button>
        </el-tooltip>
        <el-tooltip :content="$t('common.delete')" placement="top" :show-after="300">
          <el-button size="small" text type="danger" @click="onDelete(w)"
            ><el-icon><Delete /></el-icon
          ></el-button>
        </el-tooltip>
      </div>
    </div>
  </div>
```

```
      <div v-else-if="loadError" style="padding: 20px 0; text-align: center; color: var(--color-
    text-secondary)">
        {{ $t('common.loadFailed') }} <el-button text size="small" @click="load">{{ $t('common.r
    etry') }}</el-button>
      </div>
      <el-empty v-else :description="$t('wish.noData')" />
    </div>
    <el-dialog v-model="editor.visible" append-to-body :title="editor.form.id ? $t('wish.edit')
    : $t('wish.add')" width="440px">
      <el-form :model="editor.form" label-position="top">
        <el-form-item :label="$t('wish.name')">
          <el-input v-model="editor.form.title" :placeholder="$t('wish.titlePlaceholder')" />
        </el-form-item>
        <el-form-item :label="$t('wish.reason')">
          <el-input v-model="editor.form.reason" type="textarea" :rows="2" :placeholder="$t('wis
    h.reasonPlaceholder')" />
        </el-form-item>
        <el-form-item :label="$t('wish.category')">
          <el-select
            v-model="editor.cats"
            multiple
            filterable
            allow-create
            default-first-option
            :placeholder="$t('wish.categoryPlaceholder')"
            style="width: 100%"
          >
            <el-option v-for="c in CATEGORIES" :key="c" :label="c" :value="c" />
          </el-select>
        </el-form-item>
      </el-form>
      <template #footer>
        <el-button @click="editor.visible = false">{{ $t('common.cancel') }}</el-button>
        <el-button type="primary" :loading="saving" @click="onSave">{{ $t('common.save') }}</el-
    button>
      </template>
    </el-dialog>
  </div>
</template>
<script setup>
// 愿望单:愿望卡片网格 + 状态 tab;标记实现/放弃走 PUT status
import { computed, onMounted, reactive, ref } from 'vue'
import { useI18n } from 'vue-i18n'
import { Edit, Delete } from '@element-plus/icons-vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import { wishApi } from '@/api'
import { useUserStore } from '@/stores/user'
import { dictText } from '@/utils/dict'
import Breadcrumb from '@/components/Breadcrumb.vue'
import PageToolbar from '@/components/PageToolbar.vue'
```

```
const { t } = useI18n()
const userStore = useUserStore()
const CATEGORIES = ['生日礼物', '家庭用品', '旅行', '数码设备', '美食', '其他']
const loading = ref(false)
const loadError = ref(false)
const saving = ref(false)
const list = ref([])
const filter = ref('all')
const editor = reactive({ visible: false, form: {}, cats: [] })
const statusText = (s) => dictText(t, 'wishStatus', s)
const statusTag = (s) => ({ PENDING: '', ACHIEVED: 'success', ABANDONED: 'info' })[s] || 'info'
const filtered = computed(() => {
  if (filter.value === 'all') return list.value
  return list.value.filter((w) => (filter.value === 'done' ? w.status === 'ACHIEVED' : w.status
    === 'PENDING'))
})
const load = async () => {
  loading.value = true
  loadError.value = false
  try {
    list.value = await wishApi.list()
  } catch (e) {
    loadError.value = true
  } finally {
    loading.value = false
  }
}
const openEditor = (w) => {
  editor.visible = true
  editor.form = w
    ? { id: w.id, title: w.title, reason: w.reason || '', category: w.category || '' }
    : { id: null, title: '', reason: '', category: '' }
  editor.cats = editor.form.category ? editor.form.category.split(',') : []
}
const onSave = async () => {
  if (!editor.form.title?.trim()) return ElMessage.warning(t('wish.titleRequired'))
  saving.value = true
  try {
    const { id, ...data } = editor.form
    data.category = editor.cats.join(',')
    if (id) await wishApi.update(id, data)
    else await wishApi.create(data)
    ElMessage.success(t('common.success'))
    editor.visible = false
    await load()
  } finally {
    saving.value = false
  }
}
const onSetStatus = async (w, status) => {
```

```
  await wishApi.update(w.id, { status })
  ElMessage.success(status === 'ACHIEVED' ? t('wish.achievedMsg') : status === 'ABANDONED' ? t('
    wish.abandonedMsg') : t('wish.restoredMsg'))
  await load()
}
const onDelete = async (w) => {
  await ElMessageBox.confirm(t('wish.deleteConfirm', { title: w.title }), t('common.deleteConfir
    m'), {
    type: 'warning',
    closeOnClickModal: true,
  })
  await wishApi.remove(w.id)
  ElMessage.success(t('common.deleted'))
  await load()
}
onMounted(load)
</script>
<style scoped>
/* 愿望卡片网格:状态/理由/分类/操作 */
.wish-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 14px;
}
.wish-card {
  padding: 16px 18px;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
.wish-card.st1 {
  opacity: 0.65;
}
.wish-card.st1 .wish-title {
  text-decoration: line-through;
}
.wish-card.st2 {
  opacity: 0.45;
}
.wish-top {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 8px;
}
.wish-title {
  font-weight: 600;
}
.wish-reason {
  color: #666;
```

```
  font-size: 13px;
}
.wish-cats {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
}
.wish-meta {
  color: #999;
  font-size: 12px;
  display: flex;
  justify-content: space-between;
}
.wish-actions {
  display: flex;
  gap: 6px;
  flex-wrap: wrap;
}
.wish-actions :deep(.el-button.is-text) {
  padding: 5px 6px;
}
.wish-actions :deep(.el-button.is-text + .el-button.is-text) {
  margin-left: 4px;
}
@media (max-width: 768px) {
  .wish-actions .el-button {
    flex: 1;
    min-width: 60px;
  }
  .wish-meta {
    flex-direction: column;
    gap: 4px;
  }
}
</style>
// ===== frontend/src/workers/regex.worker.js =====
// 正则执行的限时沙箱:主线程只负责交互,匹配放在 Worker 里跑,超时由页面 terminate。
import { executeRegex } from '@/utils/tools/regexTool'
self.onmessage = (event) => {
  const { id, input, options, locale } = event.data || {}
  try {
    self.postMessage({ id, result: executeRegex(input, options, { locale }) })
  } catch (error) {
    self.postMessage({
      id,
      result: { ok: false, error: { code: 'workerError', message: String(error?.message || error
    ) } },
    })
  }
}
```